

Universidad de La Habana
Facultad de Matemática y Computación



Sistema web para el autorreporte de Eventos Supuestamente Atribuibles a la Vacunación o Inmunización (ESAVI) en el Instituto Finlay de Vacunas (IFV)

Autor:

Jabel Resendiz Aguirre

Tutores:

MsC. Joanna Campbell Amos

MsC. Aracelys García Armenteros

Trabajo de Diploma
presentado en opción al título de
Licenciado en Ciencia de la Computación

31 de mayo de 2026

github.com/JabelResendiz/thesis

Capítulo 1

Detalles de Implementación

1.1. Introducción

Este capítulo describe los principales aspectos relacionados con la implementación del sistema web desarrollado para el autorreporte y la gestión operativa de ESAVI. El contenido aborda las tecnologías y herramientas incorporadas durante el desarrollo, la organización estructural de la solución y los mecanismos implementados tanto en el backend como en el frontend.

La implementación materializa las decisiones arquitectónicas definidas en el capítulo anterior mediante una arquitectura modular basada en los principios de Clean Architecture y Domain-Driven Design. La solución integra mecanismos de autenticación, comunicación asíncrona, persistencia de datos y validación de información, con el propósito de garantizar escalabilidad, mantenibilidad y seguridad.

Además, el capítulo expone los principales componentes funcionales de la plataforma, incluyendo el registro de reportes de ESAVI, la gestión de revisiones médicas, la asignación de responsables y el sistema de notificaciones. Finalmente, el contenido aborda elementos asociados al despliegue y la organización técnica de la aplicación.

1.2. Tecnologías y herramientas utilizadas

1.2.1. Tecnologías del backend

C# y .NET 8. El backend se desarrolló en C# sobre .NET 8, un framework de alto rendimiento que ofrece compilación nativa, inyección de dependencias integrada y un ecosistema maduro de bibliotecas para construir APIs REST. La documentación oficial de Microsoft establece que .NET 8 es la versión LTS (Long Term Support) recomendada para aplicaciones en producción [27].

Entity Framework Core. La persistencia de datos emplea Entity Framework Core como ORM (Object-Relational Mapper), que permite trabajar con la base de datos mediante objetos de C# y expresiones LINQ sin escribir consultas SQL manualmente [28].

1.2.2. Tecnologías de la interfaz de usuario

React con TypeScript. La interfaz de usuario se construye como una Single Page Application (SPA) en React, una biblioteca de JavaScript que permite crear interfaces interactivas mediante componentes reutilizables. TypeScript añade tipado estático sobre JavaScript, lo que reduce errores en tiempo de desarrollo y mejora la mantenibilidad del código.

1.2.3. Sistema gestor de bases de datos

MySQL. El sistema utiliza MySQL como motor de base de datos relacional. La documentación oficial de MySQL establece que su arquitectura de almacenamiento basada en tablas y su soporte para claves foráneas e índices lo hacen adecuado para aplicaciones que requieren integridad referencial estricta [29]. Esta característica resultó determinante para un dominio como el de farmacovigilancia, donde entidades como reportes, sujetos vacunados, revisores y vacunas deben mantener relaciones consistentes.

1.2.4. Herramientas de infraestructura y despliegue

Docker. La contenedorización de la aplicación empleó **Docker** para garantizar la consistencia entre los entornos de desarrollo, pruebas y producción. La documentación oficial de Docker establece que los contenedores empaquetan el código junto con todas sus dependencias, lo que asegura que el software funcione de manera uniforme en cualquier infraestructura [30]. Esta característica elimina el clásico problema de “en mi máquina funciona” y facilita la distribución del proyecto.

Railway. El despliegue de la API, la base de datos MySQL y los servicios externos como RabbitMQ recayó en **Railway**, una plataforma como servicio que permite aprovisionar infraestructura en la nube y conectar el despliegue directamente con un repositorio de GitHub [31]. Railway gestiona de forma automática las variables de entorno, los volúmenes de datos y la exposición de puertos, y activa una nueva construcción y publicación con cada cambio en la rama principal, lo que reduce significativamente el esfuerzo operativo durante el desarrollo [32].

Vercel. Para el *frontend*, el despliegue recayó en **Vercel**, una plataforma optimizada para aplicaciones web modernas que ofrece integración continua con repositorios

de GitHub y distribución global mediante una red de entrega de contenido [33].

Es importante señalar que tanto Railway como Vercel constituyen soluciones de desarrollo y demostración, adoptadas exclusivamente para facilitar que el equipo de investigadores del Instituto Finlay de Vacunas (IFV) y los tutores del proyecto pudieran observar los avances de forma remota y en tiempo real. La arquitectura del sistema está diseñada para migrar y contenerizar en servidores físicos dedicados del IFV cuando el proyecto transite a la etapa de producción, garantizando así el control institucional sobre los datos y la infraestructura.

1.2.5. Herramientas de apoyo al desarrollo

Visual Studio Code. El desarrollo del proyecto transcurrió en **Visual Studio Code**, con las extensiones oficiales para C# y Docker. Microsoft recomienda este entorno para el desarrollo en .NET, ya que ofrece una integración completa con el SDK y la CLI de .NET [34]. Dicha integración permite crear, compilar y ejecutar aplicaciones directamente desde el editor, lo que agiliza el flujo de trabajo [35].

Git y GitHub. El control de versiones recayó con **Git** y el alojamiento del código en **GitHub**. GitHub proporciona un historial completo de cambios donde es posible consultar qué modificaciones ocurrieron, quién las hizo y por qué [36]. Al tratarse de un sistema distribuido, cada copia local del repositorio contiene el historial completo del proyecto, lo que permite recuperar cualquier estado anterior del código sin depender del servidor central [37].

Swagger (Swashbuckle). Para documentar y probar la API, el proyecto incorporó **Swagger** a través de la librería Swashbuckle, que genera automáticamente una interfaz web con todos los endpoints, sus parámetros y permite ejecutar peticiones de prueba directamente desde el navegador [38].

1.3. Implementación de la arquitectura

Una vez definida la arquitectura del sistema en la sección ??, este apartado describe cómo se materializaron las decisiones de diseño en código. La implementación sigue la estructura en capas propuesta por la arquitectura limpia: dominio, aplicación, infraestructura y presentación. Por cada capa se presentan los patrones utilizados, las tecnologías que los materializan y los fragmentos de código que ilustran las decisiones más relevantes.

1.3.1. Capa de Dominio

La capa de dominio constituye el núcleo del sistema y no depende de ningún otro proyecto de la solución ni de paquetes externos. Contiene las entidades de negocio,

los objetos de valor, los enumerados, los eventos de dominio y las reglas intrínsecas del dominio de farmacovigilancia. Su independencia garantiza que los cambios en la infraestructura o en la presentación no afecten la lógica central del negocio.

Entidades y agregados

El diseño de las entidades parte del modelo de base de datos definido en el Capítulo 3. Cada tabla se modela como una clase de C# que hereda de una jerarquía de clases base diseñada para unificar propiedades comunes y permitir la creación de interfaces genéricas en la capa de aplicación.

La jerarquía de herencia parte de **BasicEntity**, una clase abstracta que define las marcas de tiempo comunes a todas las entidades del sistema:

```
1 public abstract class BasicEntity
2 {
3     public DateTime CreatedAt { get; set; }
4     public DateTime UpdatedAt { get; set; }
5 }
```

Ejemplo de código 1.1: Clase base BasicEntity con marcas de tiempo

Para contemplar entidades con distintos tipos de identificador, el dominio define la interfaz genérica **IEntity<TId>**:

```
1 public interface IEntity<TId>
2 {
3     TId Id { get; init; }
4 }
```

Ejemplo de código 1.2: Interfaz genérica para identificadores de entidad

A partir de esta interfaz, el dominio proporciona dos clases abstractas especializadas. La clase **GuidEntity** constituye la norma para la mayoría de las entidades del sistema y utiliza un **Guid** como identificador:

```
1 public abstract class GuidEntity : BasicEntity, IEntity<Guid>
2 {
3     public Guid Id { get; init; } = new Guid();
4 }
```

Ejemplo de código 1.3: Clase GuidEntity para entidades con identificador Guid

Para entidades de catálogo que requieren un identificador entero —como las listas de provincias y municipios, cuyos registros son pocos, estables y no requieren las propiedades de seguridad de un **Guid**—, existe la clase **CatalogEntity**.

```
1 public abstract class CatalogEntity : BasicEntity, IEntity<int>
2 {
3     public int Id { get; init; }
```

```
4 }
```

Ejemplo de código 1.4: Clase `CatalogEntity` para entidades con identificador entero

La elección de `Guid` como tipo de identificador para las entidades principales responde a criterios de seguridad y escalabilidad. La documentación oficial de Microsoft establece que un `Guid` (Globally Unique Identifier) es un número pseudoaleatorio de 128 bits con una probabilidad de colisión tan baja que resulta estadísticamente insignificante [39]. A diferencia de los identificadores enteros autoincrementales, un `Guid` no revela la cantidad de registros en una tabla y no es predecible, lo que refuerza la seguridad de la aplicación. Esta propiedad permite, por ejemplo, exponer el identificador en una URL sin riesgo de que un usuario malintencionado itere sobre valores consecutivos.

Gracias a que `BasicEntity` unifica las propiedades comunes `CreatedAt` y `UpdatedAt`, la capa de aplicación puede definir interfaces genéricas que operan sobre cualquier entidad del sistema sin necesidad de implementaciones específicas por cada tipo. El siguiente fragmento, extraído de la capa de aplicación, ilustra esta ventaja:

```
1 public interface IGenericRepository<T> where T : BasicEntity
2 {
3     Task<T> GetByIdAsync<TId>(TId elementId);
4     Task DeleteByIdAsync<TId>(TId elementId);
5 }
```

Ejemplo de código 1.5: Interfaz genérica que opera sobre cualquier entidad que herede de `BasicEntity`

La restricción `where T : BasicEntity` garantiza en tiempo de compilación que la interfaz solo reciba entidades que posean las marcas de tiempo `CreatedAt` y `UpdatedAt`, independientemente del tipo de su identificador. Así, el sistema puede incorporar nuevas entidades —ya sea con `Guid` o con `int` como identificador— sin modificar las interfaces genéricas existentes, en cumplimiento del Principio de Abierto/Cerrado de SOLID.

Entidades User y Role Para la gestión de usuarios y roles, el proyecto emplea las clases `IdentityUser` y `IdentityRole` del paquete `Microsoft.AspNetCore.Identity`. Esta decisión evita implementar desde cero funcionalidades críticas como el almacenamiento seguro de contraseñas, la generación de tokens de verificación y el bloqueo de cuentas tras intentos fallidos.

La entidad `User` hereda de `IdentityUser<Guid>` y extiende el modelo con los campos requeridos por el dominio:

```
1 public class User : IdentityUser<Guid>
2 {
3     public Guid RoleId { get; set; }
```

```
4 public DateTime CreatedAt { get; set; }
5 public DateTime UpdatedAt { get; set; }
6 }
```

Ejemplo de código 1.6: Entidad User que hereda de IdentityUser<Guid>

Aunque la clase derivada solo declara tres propiedades adicionales, la herencia de IdentityUser<Guid> aporta un conjunto completo de campos que cubren todas las necesidades de gestión de identidad [40]. **UserName** almacena el nombre de usuario único para el inicio de sesión. **Email** contiene el correo electrónico empleado para notificaciones y verificación de cuenta. **PasswordHash** guarda el resultado del algoritmo de hashing y nunca contiene la contraseña en texto plano. **PhoneNumber** registra el teléfono para autenticación de dos factores o recuperación de cuenta. **EmailConfirmed** indica si el usuario verificó su correo electrónico. **LockoutEnd** establece la fecha y hora hasta la cual la cuenta permanece bloqueada tras superar el máximo de intentos fallidos. **AccessFailedCount** lleva el contador de intentos fallidos consecutivos. **SecurityStamp** almacena un valor aleatorio que cambia al modificar credenciales, lo que invalida automáticamente sesiones y tokens anteriores.

La entidad Role hereda de IdentityRole<Guid> y mantiene la misma coherencia en el tipo de identificador:

```
1 public class Role : IdentityRole<Guid>{ }
```

Ejemplo de código 1.7: Entidad Role que hereda de IdentityRole<Guid>

La herencia de IdentityRole<Guid> proporciona las propiedades **Name** (nombre único del rol) y **NormalizedName** (versión normalizada para búsquedas). La relación entre usuarios y roles sigue una restricción de uno a muchos: cada usuario posee un único rol mediante la clave foránea **RoleId** en la entidad **User**, mientras que un mismo rol puede agrupar a múltiples usuarios. Esta decisión simplifica la administración de permisos y refleja los perfiles definidos en el diseño del sistema (administrador, médico evaluador y jefe de vacunación municipal).

El uso de Guid como tipo de identificador en IdentityUser y IdentityRole mantiene la coherencia con la clase GuidEntity definida para el resto de las entidades del sistema, lo que permite que las tablas de identidad compartan el mismo tipo de clave primaria que las tablas de negocio.

Mecanismo de hashing de contraseñas La documentación oficial de Microsoft describe que el PasswordHasher de Identity implementa el algoritmo PBKDF2 (Password-Based Key Derivation Function 2), estándar del RFC 2898 que deriva una clave segura a partir de una contraseña [41]. El algoritmo toma la contraseña del usuario, la combina con un *salt* criptográficamente aleatorio de 128 bits y aplica

repetidamente la función HMAC-SHA256¹ durante 100,000 iteraciones como valor predeterminado [43]. Cada iteración adicional duplica el costo computacional de un ataque por fuerza bruta sin afectar perceptiblemente el tiempo de verificación para un usuario legítimo. El formato del hash generado produce una cadena con la siguiente estructura [44]:

`Base64(salt).Base64(subkey)`

Donde el *salt* es un valor aleatorio único por cada contraseña y el *subkey* es el resultado de aplicar PBKDF2 durante el número configurado de iteraciones.

Este mecanismo garantiza tres propiedades de seguridad esenciales:

- **Unicidad del hash:** dos usuarios con la misma contraseña producen hashes completamente diferentes porque el *salt* generado para cada uno es distinto. Esto impide que un atacante compare hashes para identificar contraseñas idénticas dentro de una base de datos comprometida [45].
- **Irreversibilidad computacional:** PBKDF2 es una función unidireccional. El elevado número de iteraciones convierte el cálculo del hash en un proceso deliberadamente costoso en tiempo de CPU, lo que ralentiza significativamente cualquier intento de ataque por fuerza bruta incluso si el atacante posee hardware especializado [43].
- **Invalidación de sesiones previas:** la propiedad `SecurityStamp`, generada aleatoriamente y modificada cada vez que el usuario cambia su contraseña, se incluye en la generación de tokens de autenticación. Si un atacante obtiene credenciales antiguas, el cambio de `SecurityStamp` invalida inmediatamente cualquier sesión o token previo [46].

La verificación de la contraseña durante el inicio de sesión se realiza internamente mediante el método `VerifyHashedPassword`, que recalcula el hash con el *salt* almacenado y compara los resultados en tiempo constante, sin que la aplicación necesite conocer el texto plano de la contraseña en ningún momento [43].

¹HMAC-SHA256: código de autenticación de mensajes basado en hash [42]. Funciona aplicando de forma anidada el algoritmo SHA-256 a la combinación de una clave secreta (o *salt*) con el mensaje. Internamente, la clave se procesa mediante una operación XOR con una secuencia de relleno interna (*ipad*) antes de aplicar el primer hash al mensaje; posteriormente, el resultado se combina mediante XOR con una secuencia externa (*opad*) y se somete a un segundo hash SHA-256, garantizando simultáneamente la integridad y autenticidad del resultado.

Generación de tokens para verificación de identidad El paquete Identity incluye proveedores de tokens integrados que generan cadenas criptográficamente seguras para distintos propósitos. El sistema emplea estos tokens durante el proceso de registro para verificar que un usuario controla la dirección de correo electrónico declarada y para permitir el establecimiento seguro de su contraseña inicial. La documentación oficial explica que el proveedor predeterminado `DataProtectorTokenProvider` utiliza internamente la API de Protección de Datos de ASP.NET Core, la cual aplica cifrado autenticado (AES-256-CBC² con HMAC-SHA256) sobre los datos del token [48]. Esto garantiza tres propiedades de seguridad esenciales: el token no revela información del usuario si es interceptado, cualquier modificación invalida la firma y provoca el rechazo del servidor, y un atacante no puede generar un token válido sin acceder a las claves de protección del servidor [49].

El token posee una vigencia predeterminada de 24 horas, controlada por la propiedad `TokenLifespan` de `DataProtectionTokenProviderOptions`, transcurrida la cual expira automáticamente [50]. La validación ocurre en el servidor sin necesidad de consultar una tabla de tokens pendientes, lo que simplifica la arquitectura y elimina la necesidad de limpiar tokens expirados. El caso de uso concreto que orquesta la generación y validación de estos tokens durante el registro de médicos evaluadores se detalla en la sección dedicada a la capa de aplicación.

Configuración de seguridad adicional Además de los valores predeterminados, el proyecto configura el número de iteraciones del algoritmo PBKDF2 a 600,000 mediante la clase `PasswordHasherOptions`:

```
1 builder.Services.Configure<PasswordHasherOptions>(options =>
2 {
3     options.IterationCount = 600_000;
4 });
```

Ejemplo de código 1.8: Configuración de iteraciones de hashing en Program.cs

Este ajuste sigue las recomendaciones actuales de OWASP para el algoritmo PBKDF2-SHA256, que establecen un mínimo de 600,000 iteraciones para aplicaciones desplegadas en entornos de producción [45]. El incremento respecto al valor predeterminado de 100,000 iteraciones aumenta el costo computacional de cada intento de verificación de contraseña en un factor de seis, lo que dificulta aún más los ataques de diccionario sin afectar perceptiblemente el tiempo de inicio de sesión de un usuario legítimo.

²AES-256-CBC (Advanced Encryption Standard con modo Cipher Block Chaining y clave de 256 bits): algoritmo de cifrado simétrico estandarizado por el Instituto Nacional de Estándares y Tecnología (NIST, por sus siglas en inglés), agencia del gobierno de Estados Unidos. Transforma datos legibles en texto cifrado usando una clave secreta, y el modo CBC garantiza que mensajes idénticos produzcan resultados cifrados diferentes [47].

Objetos de valor

El dominio incluye el objeto de valor `IdentityNumber`, que encapsula la validación del formato del carné de identidad cubano, la extracción de la fecha de nacimiento y el cálculo de la edad. Esta decisión responde a que varias entidades del sistema —`VaccinatedSubject`, `Reporter` y `MedicalReviewer`— contienen un carné de identidad y requieren las mismas operaciones sobre él. Al extraer esta lógica en un objeto de valor inmutable, se evita la duplicación de código y se garantiza que cualquier validación o cálculo sobre el carné se aplique de manera uniforme en todo el sistema.

Enumerados

Los enumerados representan clasificaciones propias del negocio cuyos valores no cambian durante la ejecución de la aplicación. El dominio los define mediante el tipo `enum` de C#, lo que garantiza que toda la aplicación comparta las mismas categorías sin riesgo de discrepancias entre capas. Estos tipos no constituyen entidades ni tablas en la base de datos porque sus valores son fijos y no requieren persistencia independiente.

En contraste, conceptos como `Province` y `Municipality` sí se modelan como entidades y tablas. Existe una relación entre ambos —cada municipio pertenece a una provincia— que resulta más clara de expresar mediante llaves foráneas que mediante un enumerado extenso. Además, el sistema realiza operaciones frecuentes de búsqueda y filtrado sobre estos datos, para las cuales una tabla indexada ofrece mejor rendimiento. Por último, la lista de provincias y municipios puede actualizarse ocasionalmente por cambios administrativos, y una tabla permite modificarla sin recompilar la aplicación.

Eventos de dominio

Los eventos de dominio representan sucesos relevantes del negocio que otras partes del sistema deben conocer. El proyecto los define como clases simples con propiedades que identifican los datos del suceso y los utiliza para desencadenar el envío de notificaciones por correo electrónico sin acoplar la lógica de envío a las entidades. Esta arquitectura mantiene el dominio aislado de los detalles de infraestructura y permite agregar nuevos manejadores sin modificar las entidades ni los casos de uso.

Durante la implementación surgieron además dos entidades de infraestructura no previstas en el diseño inicial: `RefreshToken`, que permite la renovación de sesiones sin reautenticación, y `AuditLog`, que registra las operaciones realizadas sobre el sistema con fines de trazabilidad. Ambas heredan de `GuidEntity` y su definición detallada se presenta en el Anexo E.

1.3.2. Capa de Aplicación

La capa de aplicación orquesta los casos de uso del sistema. Recibe comandos y consultas desde la capa de presentación, coordina las entidades del dominio y los servicios de infraestructura, y retorna resultados transformados en DTOs. Esta capa depende exclusivamente del dominio, por lo que desconoce los detalles de la base de datos, el envío de correos o la generación de tokens.

Casos de uso y servicios de aplicación

Los casos de uso se implementan mediante servicios de aplicación que el controlador invoca a través de interfaces. Esta decisión responde al patrón de Inyección de Dependencias (Dependency Injection, DI): en lugar de que cada clase cree sus propias dependencias mediante `new`, el contenedor de DI de ASP.NET Core las suministra automáticamente al constructor. El desarrollador registra qué implementación concreta corresponde a cada interfaz, y el contenedor resuelve las dependencias en tiempo de ejecución [51].

Este patrón resuelve un problema concreto de acoplamiento. Sin DI, la capa de presentación necesitaría conocer las clases concretas de la capa de aplicación para instanciarlas. Con DI, la presentación solo declara la interfaz en su constructor, sin saber qué implementación recibe ni dónde se encuentra. Así, si la lógica de negocio cambia, solo debe modificarse la capa de aplicación; la presentación permanece intacta. Este mismo principio se aplica en todas las capas del sistema: la capa de aplicación declara interfaces de repositorio que la infraestructura implementa, y la capa de infraestructura declara interfaces de servicios externos que los proveedores concretos satisfacen. El contenedor centraliza las dependencias en métodos como `AddApplication` e `AddInfrastructure`:

```
1 // Capa de aplicación: AddApplication
2 services.AddScoped<IReportCommandService, ReportCommandService>();
3 services.AddScoped<IReportQueryService, ReportQueryService>();
4 services.AddScoped<IIdentityService, IdentityService>();
5
6 // Capa de infraestructura: AddInfrastructure
7 services.AddScoped<IUnitOfWork, UnitOfWork>();
8 services.AddScoped<IJwtTokenGenerator, JwtTokenGenerator>();
9 services.AddScoped<IEmailService, EmailJsService>();
```

Ejemplo de código 1.9: Registro de servicios en el contenedor de DI

El siguiente fragmento muestra cómo el controlador recibe `IIdentityService` sin conocer su implementación, y cómo el servicio de aplicación recibe a su vez `IJwtTokenGenerator` y `IUnitOfWork` bajo el mismo principio:

```
1 // Capa de presentación: solo conoce la interfaz
```

```

2 public class AuthenticationController : ControllerBase
3 {
4     private readonly IIdentityService _identityService;
5
6     public AuthenticationController(IIdentityService identityService
7     )
8     {
9         _identityService = identityService;
10    }
11
12    // Capa de aplicación: recibe interfaces, no implementaciones
13    public class IdentityService : IIdentityService
14    {
15        private readonly IJwtTokenGenerator _jwtTokenGenerator;
16        private readonly IUnitOfWork _unitOfWork;
17
18        public IdentityService(
19            IJwtTokenGenerator jwtTokenGenerator,
20            IUnitOfWork unitOfWork)
21        {
22            _jwtTokenGenerator = jwtTokenGenerator;
23            _unitOfWork = unitOfWork;
24        }
25    }

```

Ejemplo de código 1.10: Inyección de abstracciones en todas las capas

El sistema aplica además el Principio de Separación de Comandos y Consultas (CQS), propuesto por Bertrand Meyer, que establece que una operación debe ser un comando —modifica el estado sin retornar datos— o una consulta —retorna datos sin modificar el estado—, pero nunca ambas cosas [52]. Llevado a la arquitectura, cada recurso del dominio define dos interfaces: una para comandos y otra para consultas. El siguiente fragmento muestra esta separación para el recurso de reportes:

```

1 // Comandos: modifican el estado
2 public interface IReportCommandService
3 {
4     Task<Guid> CreatePublicReportAsync(PublicAefiReportDto dto);
5     Task CompleteReviewAsync(CompleteReviewDto dto);
6 }
7
8 // Consultas: retornan datos sin modificar
9 public interface IReportQueryService
10 {
11     Task<PagedResultDto<ReportDto>> GetReportsAsync(PagedRequestDto
12     paged);
13     Task<ReportDto> GetReportByNotificationNumber(string
14     notificationNumber);

```

13 }

Ejemplo de código 1.11: Separación CQS en interfaces de reportes

Esta separación no alcanza la complejidad de un CQRS completo —ambos servicios comparten el mismo modelo de dominio y la misma base de datos—, pero permite que cada interfaz evolucione según sus necesidades específicas.

Los servicios de aplicación coordinan los repositorios y el Unit of Work para ejecutar los casos de uso. El siguiente fragmento, extraído de `IdentityService`, muestra cómo un caso de uso orquesta múltiples dependencias dentro de una misma transacción:

```
1 public async Task<UserResponseDTO> LoginUserAsync(LoginUserDto
   loginDto)
2 {
3     var savedUser = await _identityManager.CheckCredentialsAsync(
4         loginDto.Email, loginDto.Password)
5         ?? throw new ArgumentException("Invalid credentials");
6
7     var accessToken = await _jwtTokenGenerator.GenerateToken(
8         savedUser);
9     var refreshToken = GenerateRefreshToken();
10
11     await _unitOfWork.GetRepository<RefreshToken>().CreateAsync(
12         new RefreshToken
13         {
14             Token = refreshToken,
15             ExpiresAt = DateTime.UtcNow.AddDays(7),
16             UserId = savedUser.Id
17         });
18
19     await _unitOfWork.CompleteAsync();
20
21     return new UserResponseDTO { ... };
```

Ejemplo de código 1.12: Orquestación de dependencias en un caso de uso

Mapeo y transformación de datos

La capa de aplicación define DTOs (Data Transfer Objects, objetos de transferencia de datos) para mover información entre la API y el dominio. Un DTO es un objeto plano que contiene solo propiedades, sin lógica de negocio, diseñado para transportar datos en un formato adecuado para el cliente. Las entidades del dominio no se exponen fuera de esta capa, lo que protege al núcleo del sistema de cambios en los formatos de intercambio y evita la exposición accidental de propiedades internas.

El sistema emplea AutoMapper para convertir entidades en DTOs y viceversa, aplicando el patrón Mapper [53]. AutoMapper permite definir reglas de transformación en clases llamadas perfiles, que heredan de **Profile** y centralizan las conversiones entre tipos. Cada perfil declara cómo se asigna cada propiedad de un tipo origen a un tipo destino, incluyendo propiedades anidadas que requieren lógica adicional:

```
1 public class AefiReportProfile : Profile
2 {
3     public AefiReportProfile()
4     {
5         CreateMap<AefiReport, ReportDto>()
6             .ForMember(dest => dest.SubjectName,
7                 opt => opt.MapFrom(src => src.VaccinatedSubject.
8                     FullName))
9             .ForMember(dest => dest.VaccineNames,
10                 opt => opt.MapFrom(src => src.Vaccinations
11                     .Select(v => v.Lot.Vaccine.Name)));
12     }
13 }
```

Ejemplo de código 1.13: Perfil de mapeo con AutoMapper

En este ejemplo, **SubjectName** no existe directamente en **AefiReport**, sino que se obtiene navegando la relación hacia **VaccinatedSubject**. AutoMapper resuelve esta navegación automáticamente y genera la consulta SQL correspondiente.

Para las consultas, el sistema aprovecha el método **ProjectTo<T>** de AutoMapper. A diferencia de un mapeo en memoria —que cargaría la entidad completa para luego transformarla—, **ProjectTo** traduce la proyección directamente a SQL, de modo que solo las columnas necesarias viajan desde la base de datos hasta la aplicación:

```
1 var items = await _unitOfWork.GetRepository<AefiReport>()
2     .GetPaged(query, skip, take)
3     .ProjectTo<ReportDto>(_mapper.ConfigurationProvider)
4     .ToListAsync();
```

Ejemplo de código 1.14: Proyección directa a DTO con ProjectTo

Esta técnica mejora el rendimiento de las consultas al evitar la carga completa de las entidades y sus relaciones, y al delegar la selección de columnas al motor de base de datos.

Validaciones y manejo de solicitudes

El sistema aplica dos niveles de validación sobre cada solicitud entrante, en momentos distintos del procesamiento y con propósitos complementarios.

El primer nivel opera sobre los DTOs mediante atributos de validación de .NET. Estos atributos verifican condiciones simples —obligatoriedad, longitud máxima, formato de correo electrónico o número de teléfono— antes de que la solicitud alcance

el servicio de aplicación. ASP.NET Core ejecuta esta validación automáticamente al construir el DTO desde el cuerpo de la solicitud, gracias al atributo `[ApiController]`. Si algún atributo falla, el middleware devuelve un código 400 con los campos que incumplieron la validación. El siguiente fragmento muestra un DTO con atributos de validación estándar y personalizados:

```
1 public class VaccinatedSubjectDto
2 {
3     [Required(ErrorMessage = "Full name is required.")]
4     [StringLength(150, MinimumLength = 1)]
5     public string FullName { get; set; } = string.Empty;
6
7     [Required(ErrorMessage = "Identity number is required.")]
8     [IdentityNumberValidation]
9     public string IdentityNumber { get; set; } = string.Empty;
10
11     // el resto de atributos
12 }
```

Ejemplo de código 1.15: DTO con atributos de validación

Los atributos personalizados `IdentityNumberValidation` y `EmailValidation`, definidos en la capa de aplicación, heredan de `ValidationAttribute` y encapsulan reglas reutilizables sin duplicar código en múltiples DTOs:

```
1 public class IdentityNumberValidationAttribute : ValidationAttribute
2 {
3     protected override ValidationResult? IsValid(object? value,
4         ValidationContext validationContext)
5     {
6         // valida que tenga formato de 11 dígitos y coherencia
7         return ValidationResult.Success;
8     }
9 }
```

Ejemplo de código 1.16: Atributo de validación personalizado

El segundo nivel opera dentro del servicio de aplicación, una vez que la solicitud superó las validaciones de formato. Este nivel aplica reglas de negocio que requieren consultar la base de datos o comparar datos de múltiples fuentes: verificar que una vacuna exista, que un lote pertenezca a la vacuna declarada, o que la fecha de administración de la vacuna no sea posterior a la fecha de inicio de los síntomas. Para ello, el sistema define la interfaz `IReportValidator<T>` y la inyecta en los servicios de aplicación mediante el contenedor de dependencias:

```
1 public class ReportCommandService : IReportCommandService
2 {
3     // constructor de la clase y variables privadas
4 }
```

```
5     public async Task<CreateReportResponseDto>
6     CreatePublicReportAsync(
7         PublicAefiReportDto reportDto)
8     {
9         // Ejecuta todos los validadores registrados para este tipo
10        de DTO
11        foreach (var validator in _validators)
12            await validator.ValidateAsync(reportDto);
13
14        // Continúa con la creación del reporte...
15    }
```

Ejemplo de código 1.17: Inyección de validadores en el servicio de aplicación

El contenedor de dependencias resuelve automáticamente todas las implementaciones de `IReportValidator<T>` registradas y las inyecta como una colección. Esto permite que el servicio ejecute todos los validadores sin conocer sus tipos concretos, en cumplimiento del Principio de Abierto/Cerrado: agregar una nueva regla de validación solo requiere implementar la interfaz y registrarla en el contenedor, sin modificar el servicio de aplicación.

Este mismo patrón de validación en segundo nivel aplica a otras reglas de negocio que operan sobre datos ya persistidos. Un ejemplo es la expiración de asignaciones, implementada en `IAssignmentExpirationService`. Esta regla materializa RN-07 y RN-08 (Capítulo ??): determina la prioridad del reporte a partir del evento adverso más grave que contenga y aplica el plazo de evaluación correspondiente —24 horas para prioridad alta, 5 días para prioridad media y 7 días para prioridad baja—. El servicio consulta las asignaciones pendientes, compara su fecha de asignación con el plazo según la prioridad del reporte asociado y marca como expiradas aquellas que lo superan.

La capa de aplicación extiende el principio de inversión de dependencias a los servicios de infraestructura que necesita para completar sus casos de uso. Para acceder a los datos, la aplicación declara interfaces de repositorio como `IGenericRepository<T>` y `IUserRepository`, cuyas implementaciones residen en la capa de infraestructura. Para enviar notificaciones, declara `IEmailService` sin saber si el envío utiliza un servicio externo, un servidor SMTP o cualquier otro mecanismo. Para la autenticación, declara `IJwtTokenGenerator` e `IIdentityManager` sin conocer los algoritmos de firma ni las claves secretas. En cada caso, el servicio de aplicación solo conoce la interfaz y confía en que el contenedor de DI suministrará la implementación registrada en `AddInfrastructure`. Esta separación permite cambiar el proveedor de correo electrónico, el motor de base de datos o el servicio de CAPTCHA sin modificar una sola línea de la capa de aplicación.

1.3.3. Capa de Infraestructura

La capa de infraestructura implementa los contratos definidos en las capas de dominio y aplicación. Contiene el acceso a la base de datos, la generación de tokens, el envío de correos electrónicos, la mensajería asíncrona y la integración con servicios externos. Todas las dependencias hacia el mundo exterior residen aquí, lo que permite que el núcleo del sistema permanezca independiente de tecnologías concretas.

Persistencia de datos y configuración de Entity Framework Core

El sistema utiliza MySQL como motor de base de datos relacional. La conexión se establece mediante una cadena de conexión —que contiene el servidor, el nombre de la base de datos y las credenciales de acceso— almacenada en las variables de entorno del servidor bajo el nombre `AppDbConnectionString`. El paquete `Pomelo.EntityFrameworkCore.MySql`, un proveedor de .NET que adapta Entity Framework Core a las características específicas de MySQL [54], toma esta cadena y la utiliza para establecer la comunicación con la base de datos. La configuración se realiza durante el arranque de la aplicación, en un método de extensión llamado `AddInfrastructure` que centraliza el registro de todos los servicios de esta capa en el contenedor de inyección de dependencias:

```
1 var connectionString = configuration.GetConnectionString("
  AppDbConnectionString");
2 services.AddDbContext<FinlayDbContext>(options =>
3     options.UseMySQL(connectionString, ServerVersion.AutoDetect(
      connectionString)));
```

Ejemplo de código 1.18: Configuración de la conexión a MySQL

Sobre esta conexión, el contexto `FinlayDbContext` —la clase que representa la sesión con la base de datos— centraliza el acceso a todas las tablas del sistema. Las características de infraestructura de cada tabla se definen en el método `OnModelCreating` mediante la API fluida de Entity Framework Core, sin decorar las clases del dominio con atributos propios de la persistencia [28]. Este método aplica las mismas restricciones definidas en el diseño de la base de datos (Capítulo 3, Sección 3.5): nulabilidad, longitudes máximas, llaves foráneas e índices³. Además, cada relación declara un comportamiento de borrado y actualización: `Cascade` propaga la operación cuando la entidad dependiente carece de sentido sin su principal, mientras que `Restrict` la bloquea si existen registros referenciados desde otra tabla. El siguiente fragmento, correspondiente a la entidad `AefiReport`, ilustra estas decisiones:

³Un índice es una estructura de datos que permite al motor de base de datos localizar filas sin recorrer toda la tabla, de forma similar al índice alfabético de un libro. El sistema define índices sobre columnas consultadas con frecuencia.

```

1 builder.Entity<AefiReport>(entity =>
2 {
3     entity.HasKey(e => e.Id);
4     entity.Property(e => e.NotificationNumber)
5         .IsRequired()
6         .HasMaxLength(100);
7     entity.HasIndex(e => e.NotificationNumber).IsUnique();
8     entity.HasOne(r => r.VaccinatedSubject)
9         .WithMany(p => p.AefiReports)
10        .HasForeignKey(r => r.VaccinatedSubjectId)
11        .OnDelete(DeleteBehavior.Restrict);
12
13     // otros atributos y llaves foráneas
14 });

```

Ejemplo de código 1.19: Configuración de infraestructura para AefiReport

Migraciones y acceso a datos

El acceso a los datos se implementa mediante el patrón Repository, que abstrae las operaciones de persistencia detrás de interfaces definidas en la capa de aplicación. La clase `GenericRepository<T>` proporciona una implementación genérica para operaciones comunes —crear, obtener por identificador, listar, actualizar y eliminar— y aprovecha la restricción `where T : BasicEntity` para operar sobre cualquier entidad del sistema sin duplicar código:

```

1 public class GenericRepository<T> : IGenericRepository<T> where T :
   BasicEntity
2 {
3     protected readonly DbSet<T> _entity;
4
5     public virtual async Task<T> CreateAsync(T element) { ... }
6     public virtual async Task<T> GetByIdAsync<TId>(TId elementId) {
7         ... }
8     public virtual IQueryable<T> GetAll() { ... }
9     public virtual void Update(T element) { ... }
10    public virtual async Task DeleteByIdAsync<TId>(TId elementId) {
11        ... }
12 }

```

Ejemplo de código 1.20: Repositorio genérico

Cuando una entidad requiere consultas que el repositorio genérico no cubre, el sistema define repositorios especializados que heredan de `GenericRepository<T>` y añaden métodos particulares. Todas las consultas utilizan LINQ, el lenguaje de consulta integrado de C#, que Entity Framework Core traduce automáticamente a SQL. El siguiente ejemplo muestra una consulta LINQ y la sentencia SQL equivalente que se ejecuta en la base de datos:

```
1 // LINQ
2 var reportes = await _entity
3     .Where(r => r.Status == AefiReportStatus.UnderReview)
4     .OrderByDescending(r => r.ReportDate)
5     .Take(10)
6     .ToListAsync();
7
8 // SQL generado
9 SELECT TOP 10 *
10 FROM AefiReports
11 WHERE Status = 'UnderReview'
12 ORDER BY ReportDate DESC;
```

Ejemplo de código 1.21: Consulta LINQ y su traducción a SQL

El patrón Unit of Work, implementado en la clase `UnitOfWork`, coordina las operaciones de múltiples repositorios dentro de una única transacción. Su método `CompleteAsync` invoca `SaveChangesAsync` del contexto. Esto garantiza que todos los cambios realizados durante un caso de uso se persistan de forma atómica: si cualquier operación falla, ninguna se materializa en la base de datos [55].

El registro de auditoría se implementa mediante un interceptor de Entity Framework Core que hereda de `SaveChangesInterceptor`. Este interceptor se ejecuta automáticamente antes de cada `SaveChangesAsync`: recorre las entidades modificadas, serializa los valores anteriores y posteriores de cada operación, y añade los registros correspondientes a la tabla `AuditLog`. Como el interceptor opera dentro de la misma transacción, la auditoría y los cambios de negocio se persisten de forma atómica. Esta solución mantiene la transparencia para los casos de uso, que no necesitan invocar ningún servicio de auditoría.

Integración con servicios externos

El sistema integra cuatro servicios externos principales:

Friendly CAPTCHA⁴. Los endpoints públicos están expuestos a ataques automatizados que envían solicitudes directamente a la API. Como primera barrera, el sistema utiliza Friendly CAPTCHA, un servicio de verificación basado en prueba de trabajo criptográfica (proof-of-work) que no afecta la experiencia del usuario. La sección de seguridad (Apartado 1.3.5) detalla el fundamento y el flujo de verificación de este mecanismo.

Notificaciones por correo electrónico y WhatsApp. El sistema necesita notificar a los usuarios sobre eventos relevantes —confirmación de reporte, asignación a revisor, expiración de asignación— a través de canales accesibles para la población

⁴CAPTCHA (Completely Automated Public Turing test to tell Computers and Humans Apart): prueba de Turing automatizada que distingue a un humano de un ordenador.

cubana. La vía de SMS resultó inviable para esta etapa por razones de infraestructura y costo, por lo que el sistema adopta el correo electrónico y WhatsApp, las dos vías con mayor penetración en la ciudadanía cubana.

EmailJS. Para el envío de correos electrónicos, el sistema utiliza EmailJS, un servicio que permite enviar correos sin configurar un servidor SMTP propio. El servicio `EmailJsService` implementa la interfaz `IEmailService` y construye una solicitud HTTP a la API de EmailJS con los parámetros de la plantilla, el identificador del servicio y el token de acceso, todos configurados en `EmailJsSettings`:

```
1 var payload = new
2 {
3     service_id = _settings.ServiceId,
4     template_id = templateId,
5     user_id = _settings.UserId,
6     accessToken = _settings.AccessToken,
7     template_params = templateParams // diccionario que
    tiene los datos específicos de la plantilla
8 };
9
10 // Serializar y enviar la solicitud a la API de EmailJS
11 var json = JsonSerializer.Serialize(payload);
12 var content = new StringContent(json, Encoding.UTF8, "application/
    json");
13 var response = await _httpClient.PostAsync(url, content);
```

Ejemplo de código 1.22: Envío de correo mediante EmailJS

Cada tipo de notificación se corresponde con una plantilla predefinida, identificada por un `template_id`. Los datos específicos de cada plantilla viajan dentro del objeto `template_params`, que el servicio construye dinámicamente a partir de los datos recibidos desde la capa de aplicación.

OpenWA (WhatsApp). Para las notificaciones por WhatsApp, el sistema utiliza OpenWA, una API de código abierto que permite enviar mensajes mediante un servidor intermediario conectado a un número de teléfono registrado. A diferencia de la API oficial de WhatsApp Business de Meta, OpenWA no exige verificación empresarial ni tiene costo por mensaje, lo que la hace adecuada para la etapa de prototipado. El servicio que implementa esta integración sigue el mismo patrón que EmailJS: recibe los datos de la notificación desde la capa de aplicación, los transforma al formato esperado por OpenWA y envía la solicitud HTTP correspondiente. Las credenciales de acceso —URL del servidor y token de autenticación— residen en variables de entorno, sin exponerse en el código fuente.

Tanto EmailJS como OpenWA constituyen soluciones para la etapa de desarrollo y demostración. La arquitectura en capas permite migrar a servicios de producción sin modificar la lógica de negocio: EmailJS puede reemplazarse por un servidor SMTP institucional del IFV o por servicios como SendGrid o Resend con planes extensivos;

OpenWA puede migrar a la API oficial de WhatsApp Business de Meta cuando el proyecto transite a producción.

Generación de PDF. El servicio `PdfService` utiliza la librería QuestPDF para generar documentos PDF a partir de los datos de un reporte ESAVI. QuestPDF emplea una API fluida que describe la estructura del documento —páginas, tablas, texto— en código C# sin depender de plantillas HTML ni de librerías externas [56].

Comunicación asíncrona mediante mensajería

Cuando un usuario completa un reporte ESAVI, el sistema valida los datos, los persiste y envía una notificación de confirmación por correo electrónico. Aunque la operación principal se procesa correctamente, la notificación depende de un servicio externo sujeto a latencia de red o indisponibilidad temporal. Si el envío se ejecuta dentro del mismo hilo que atiende la solicitud HTTP, el usuario percibe esa demora o nunca recibe la confirmación a pesar de que su reporte se guardó exitosamente.

Las colas de mensajes resuelven este problema desacoplando la operación principal de sus efectos secundarios. El caso de uso deposita un mensaje en la cola y retorna de inmediato al usuario; el mensaje se procesa en segundo plano. Si el procesamiento falla, el mensaje permanece en la cola y puede reintentarse sin afectar al usuario.

El sistema utiliza RabbitMQ como intermediario de mensajería y MassTransit como librería de integración con .NET [57]. Para mantener la capa de aplicación independiente de MassTransit, el sistema define una abstracción propia: la interfaz `IEventBus`, que expone un único método `PublishAsync`. La implementación concreta, `MassTransitEventBus`, reside en la capa de infraestructura y delega en `IPublishEndpoint` de MassTransit. La configuración se centraliza en el método `AddInfrastructure`:

```
1 services.AddMassTransit(x =>
2 {
3     x.AddConsumer<MedicalReviewerConsumer>();
4     x.AddConsumer<AssignmentExpiredConsumer>();
5     x.AddConsumer<ReportConfirmationConsumer>();
6
7     x.UsingRabbitMq((context, cfg) =>
8     {
9         cfg.Host(rabbitMqUrl);
10        cfg.ConfigureEndpoints(context);
11    });
12 });
13
14 services.AddScoped<IEventBus, MassTransitEventBus>();
```

Ejemplo de código 1.23: Configuración de MassTransit con RabbitMQ

Cada consumidor implementa la interfaz `IConsumer<T>` de MassTransit, donde `T` es el tipo de evento de dominio que procesa. El siguiente fragmento muestra el patrón

de uso: el caso de uso guarda el reporte, publica el evento a través de `IEventBus` y retorna al usuario sin esperar:

```
1 await _unitOfWork.GetRepository<AefiReport>().CreateAsync(report);
2 await _unitOfWork.CompleteAsync();
3
4 await _eventBus.PublishAsync(new ReportConfirmationEvent
5 {
6     ReportNumber = report.NotificationNumber,
7     Email = reporter.Email,
8     SymptomsName = symptomNames,
9     VaccinesName = vaccinesNames,
10    ReportDate = report.ReportDate
11 });
12
13 return new CreateReportResponseDto
14 {
15     NotificationNumber = report.NotificationNumber
16 };
```

Ejemplo de código 1.24: Publicación de un evento tras crear un reporte

El método `PublishAsync` envía el evento a RabbitMQ y retorna de inmediato. En segundo plano, `ReportConfirmationConsumer` recibe el mensaje y ejecuta el envío mediante `IEmailService`. Si el envío falla, `MassTransit` aplica reintentos automáticos con intervalos crecientes, sin que el usuario perciba el error. El mismo mecanismo aplica al resto de notificaciones del sistema: asignación de un reporte a un médico revisor, expiración de una asignación no completada, nuevo reporte recibido por un jefe de vacunación municipal y correo de activación de cuenta para un revisor recién registrado.

Para las asignaciones que no se completan en el plazo establecido, un servicio en segundo plano verifica periódicamente las pendientes. Este servicio hereda de `BackgroundService`, clase base de .NET recomendada para tareas de larga duración [58].

```
1 public class AssignmentExpirationBackgroundService :
2     BackgroundService
3 {
4     ...
5     protected override async Task ExecuteAsync(CancellationToken
6         stoppingToken)
7     {
8         while (!stoppingToken.IsCancellationRequested)
9         {
10             // Crear un ámbito para acceder a servicios scoped
11             using var scope = _scopeFactory.CreateScope();
12             var service = scope.ServiceProvider
13                 .GetRequiredService<IAssignmentExpirationService>();
```

```
12
13         // Ejecutar la lógica de negocio en capa de aplicación
14         await service.ProcessExpiredAssignmentsAsync(
15             stoppingToken);
16
17         // Esperar 3 horas antes de la próxima verificación
18         await Task.Delay(TimeSpan.FromHours(3), stoppingToken);
19     }
20 }
```

Ejemplo de código 1.25: Servicio en segundo plano para expiración de asignaciones

En cada ciclo, el servicio invoca a `IAssignmentExpirationService`, el servicio de aplicación encargado de localizar las asignaciones pendientes cuyo plazo expiró, marcarlas como expiradas, reabrir los reportes asociados y publicar los eventos correspondientes a través de `IEventBus`. Las reglas de negocio que determinan los plazos de expiración según la severidad del reporte se detallan en la capa de aplicación (Apartado 1.3.2). Los consumidores reciben estos eventos y envían las notificaciones por correo electrónico al jefe de vacunación municipal correspondiente.

1.3.4. Capa de Presentación

La capa de presentación constituye el punto de entrada al sistema desde el exterior. Materializa los principios de la arquitectura limpia al depender exclusivamente de la capa de aplicación: los controladores reciben solicitudes HTTP, las transforman en comandos o consultas, y retornan respuestas estructuradas sin contener lógica de negocio.

Diseño de la API REST

El sistema expone una API REST que sigue las convenciones del protocolo HTTP. Esta decisión responde a dos razones: REST utiliza los verbos y códigos de estado estándar de HTTP, lo que facilita la integración con cualquier frontend web o móvil sin necesidad de librerías propietarias, y su modelo sin estado —cada solicitud contiene toda la información necesaria para procesarla— simplifica el escalamiento horizontal y la depuración [59]. Cada recurso del dominio —reportes, revisores médicos, asignaciones, vacunas— se identifica mediante una ruta única bajo el prefijo `/api/`. Los controladores utilizan los verbos HTTP estándar conforme a su propósito: POST para crear recursos, GET para consultarlos, PUT para modificarlos y DELETE para eliminarlos.

Las respuestas siguen una estructura uniforme. Las operaciones exitosas retornan el código HTTP correspondiente junto con un objeto JSON que contiene un mensaje

descriptivo y los datos solicitados. El siguiente fragmento, extraído del controlador de reportes, ilustra esta convención:

```
1 return StatusCode(StatusCodes.Status201Created, new
2 {
3     message = "Report successfully created",
4     data = result
5 });
```

Ejemplo de código 1.26: Respuesta estructurada en el controlador de reportes

Los códigos de estado HTTP comunican el resultado de cada operación de forma inequívoca: 201 para creación exitosa, 200 para consultas, 400 para errores de validación, 404 para recursos no encontrados y 500 para errores internos. Cada endpoint declara explícitamente los códigos que puede retornar mediante el atributo `[ProducesResponseType]`, lo que permite a Swagger documentarlos automáticamente.

Controladores y endpoints

Los controladores organizan los endpoints por recurso del dominio y actúan como fachada entre el protocolo HTTP y los casos de uso de la capa de aplicación. El sistema define controladores para autenticación, reportes, revisores médicos, asignaciones, vacunas, síntomas, provincias y centros de vacunación.

Cada controlador recibe mediante inyección de dependencias únicamente los servicios de aplicación que necesita. El controlador de reportes, por ejemplo, depende de `IReportQueryService` para consultas e `IReportCommandService` para comandos, lo que materializa el patrón CQRS desde la presentación:

```
1 [ApiController]
2 [Route("api/[controller]")]
3 public class ReportController : ControllerBase
4 {
5     private readonly IReportQueryService _reportQueryService;
6     private readonly IReportCommandService _reportCommandService;
7
8     public ReportController(
9         IReportQueryService reportQueryService,
10        IReportCommandService reportCommandService)
11     {
12         _reportQueryService = reportQueryService;
13         _reportCommandService = reportCommandService;
14     }
15 }
```

Ejemplo de código 1.27: Controlador de reportes con separación CQRS

Los atributos `[ApiController]` y `[Route]` configuran el comportamiento del controlador: el primero habilita la validación automática del modelo y la inferencia de parámetros desde el cuerpo de la solicitud; el segundo define la ruta base. Cada método expone un endpoint específico con su verbo HTTP, su política de rate limiting y sus tipos de respuesta documentados.

Documentación de la API

La documentación interactiva de la API se genera automáticamente mediante Swagger, integrado a través de la librería Swashbuckle. Swagger analiza los controladores, los modelos de solicitud y respuesta, y los atributos `[ProducesResponseType]` para producir una especificación OpenAPI completa y una interfaz web donde los desarrolladores pueden explorar los endpoints y realizar peticiones de prueba directamente desde el navegador.

La configuración se realiza en el método `AddPresentation` de la capa de API y se expone en el pipeline de la aplicación solo en entorno de desarrollo:

```
1 if (app.Environment.IsDevelopment())
2 {
3     app.UseSwagger();
4     app.UseSwaggerUI();
5 }
```

Ejemplo de código 1.28: Configuración de Swagger en el pipeline

Esta restricción garantiza que la documentación interactiva no quede expuesta en producción, donde podría revelar detalles de la API a usuarios no autorizados.

Manejo de excepciones y respuestas

El sistema centraliza el manejo de errores en el middleware `ErrorHandlingMiddleware`, que intercepta cualquier excepción no controlada que ocurra durante el procesamiento de una solicitud. Este middleware evita que los detalles de implementación —como el seguimiento de pila— lleguen al cliente, y traduce cada tipo de excepción a un código HTTP y un mensaje apropiado [60]:

```
1 public class ErrorHandlingMiddleware
2 {
3     private readonly RequestDelegate _next;
4
5     public async Task Invoke(HttpContext context)
6     {
7         try
8         {
9             await _next(context);
10        }
```

```
11         catch (Exception ex)
12         {
13             await HandleExceptionAsync(context, ex);
14         }
15     }
16
17     private static Task HandleExceptionAsync(HttpContext context,
18     Exception exception)
19     {
20         var code = HttpStatusCode.InternalServerError;
21         string message = "An unexpected error occurred.";
22
23         switch (exception)
24         {
25             case KeyNotFoundException:
26                 code = HttpStatusCode.NotFound;
27                 message = exception.Message;
28                 break;
29             case ArgumentNullException:
30             case ArgumentException:
31                 code = HttpStatusCode.BadRequest;
32                 message = exception.Message;
33                 break;
34             case InvalidOperationException:
35                 code = HttpStatusCode.BadRequest;
36                 message = exception.Message;
37                 break;
38         }
39
40         var result = JsonSerializer.Serialize(new { message });
41         context.Response.ContentType = "application/json";
42         context.Response.StatusCode = (int)code;
43         return context.Response.WriteAsync(result);
44     }
```

Ejemplo de código 1.29: Middleware de manejo de excepciones

El mapeo de excepciones distingue tres categorías: `KeyNotFoundException` retorna 404 (recurso no encontrado), `ArgumentException` y `ArgumentNullException` retornan 400 (datos de entrada inválidos), y cualquier otra excepción retorna 500 con un mensaje genérico. Esta clasificación permite al frontend interpretar el error y actuar en consecuencia sin necesidad de inspeccionar el mensaje.

Para los errores de validación del modelo, el sistema configura `ApiBehaviorOptions` en `Program.cs` con una fábrica de respuestas personalizada que retorna un objeto JSON estructurado con los campos que fallaron y los mensajes correspondientes [61]:

```
1 builder.Services.Configure<ApiBehaviorOptions>(options =>
```

```
2 {
3     options.InvalidModelStateResponseFactory = context =>
4     {
5         var errors = context.ModelState
6             .Where(x => x.Value.Errors.Count > 0)
7             .Select(x => new
8             {
9                 Field = x.Key,
10                 Errors = x.Value.Errors.Select(e => e.ErrorMessage)
11             });
12
13         return new BadRequestObjectResult(new
14         {
15             success = false,
16             status = 400,
17             message = "Validation failed",
18             errors
19         });
20     };
21 });
```

Ejemplo de código 1.30: Configuración de respuestas de validación

Esta configuración garantiza que incluso los errores detectados antes de que la solicitud llegue al controlador —como un campo requerido ausente o un tipo de dato incorrecto— sigan el mismo formato de respuesta que el resto de la API.

1.3.5. Seguridad de la aplicación

Autenticación basada en JWT y Refresh Token

El sistema emplea JSON Web Tokens (JWT) como mecanismo de autenticación sin estado, combinado con tokens de actualización (refresh tokens) almacenados en cookies `HttpOnly`. Esta combinación responde a dos objetivos contrapuestos: el token de acceso debe tener una vida útil corta para limitar el daño en caso de robo, pero el usuario no debe verse obligado a iniciar sesión repetidamente.

JSON Web Token (JWT) Un JWT es un estándar abierto definido en el RFC 7519 que permite transmitir información verificable entre dos partes mediante un token firmado digitalmente [62]. El token consta de tres secciones codificadas en Base64 y separadas por puntos: encabezado, carga útil y firma. La carga útil contiene los *claims* —afirmaciones sobre el usuario, como su identificador, nombre y rol—, mientras que la firma garantiza que el token no fue alterado desde su emisión.

El sistema genera el JWT mediante la clase `JwtTokenGenerator`, que utiliza HMAC-SHA256 como algoritmo de firma con una clave secreta simétrica almace-

nada en variables de entorno. Esta clave nunca aparece en el código fuente ni se expone al cliente.

```
1 public async Task<string> GenerateToken(User user)
2 {
3     var key = new SigningCredentials(
4         new SymmetricSecurityKey(Encoding.UTF8.GetBytes(_jwtSettings
5             .Secret)),
6         SecurityAlgorithms.HmacSha256);
7
8     var role = await _userManager
9         .GetRolesAsync(user)
10        .FirstOrDefault();
11
12    var claims = new List<Claim>
13    {
14        new Claim(JwtRegisteredClaimNames.Sub, user.Id.ToString()),
15        new Claim(JwtRegisteredClaimNames.GivenName, user.UserName!),
16        new Claim(JwtRegisteredClaimNames.Jti, Guid.NewGuid().
17            ToString()),
18        new Claim(ClaimTypes.Role, role!)
19    };
20
21    var securityToken = new JwtSecurityToken(
22        signingCredentials: key,
23        issuer: _jwtSettings.Issuer,
24        audience: _jwtSettings.Audience,
25        expires: DateTime.UtcNow.AddMinutes(_jwtSettings.
26            ExpiryMinutes),
27        claims: claims);
28
29    return new JwtSecurityTokenHandler().WriteToken(securityToken);
30 }
```

Ejemplo de código 1.31: Generación del JWT en JwtTokenGenerator

El método define cuatro claims en el token: identificador del usuario (**Sub**), nombre de usuario (**GivenName**), identificador único del token (**Jti**) y rol del usuario (**Role**). La expiración se configura en 15 minutos mediante **JwtSettings.ExpiryMinutes**.

El middleware de ASP.NET Core valida cada solicitud entrante verificando cuatro propiedades del token: que el emisor y la audiencia coincidan con los valores configurados, que la firma HMAC-SHA256 sea correcta y que el token no haya expirado. Si un atacante intercepta el JWT, puede usarlo durante un máximo de 15 minutos, transcurridos los cuales el token expira y el middleware lo rechaza automáticamente.

Refresh Token en cookie HttpOnly Para compensar la corta vida del JWT sin sacrificar seguridad, el sistema implementa refresh tokens. Un refresh token es un valor aleatorio generado criptográficamente que permite obtener nuevos tokens de acceso sin que el usuario vuelva a ingresar sus credenciales. El método `GenerateRefreshToken` utiliza la clase `RandomNumberGenerator` de .NET, una alternativa criptográficamente segura frente a generadores predecibles como `System.Random` [63].

```
1 private string GenerateRefreshToken()  
2 {  
3     using var rng = RandomNumberGenerator.Create();  
4     var randomBytes = new byte[64];  
5     rng.GetBytes(randomBytes);  
6     return Convert.ToBase64String(randomBytes);  
7 }
```

Ejemplo de código 1.32: Generación criptográfica del refresh token

El método produce 64 bytes de entropía criptográfica y los codifica en Base64. Este valor se almacena en la base de datos junto con su fecha de vencimiento y el identificador del usuario.

La seguridad de este mecanismo descansa en tres decisiones reflejadas en la configuración de la cookie que transporta el refresh token:

```
1 Response.Cookies.Append("refreshToken", authResult.RefreshToken,  
2     new CookieOptions  
3     {  
4         HttpOnly = true,  
5         Secure = true,  
6         SameSite = SameSiteMode.None,  
7         Expires = DateTime.UtcNow.AddHours(_jwtSettings.  
RefreshTokenExpiryHours)  
8     });
```

Ejemplo de código 1.33: Configuración de la cookie del refresh token

- `HttpOnly = true`: impide que JavaScript acceda al valor de la cookie, lo que neutraliza los ataques de cross-site scripting (XSS), ataque que inyecta código JavaScript malicioso en una página web legítima para robar datos del usuario[64].
- `Secure = true`: restringe la transmisión de la cookie a conexiones HTTPS, evitando su exposición en redes no cifradas.
- `SameSite = None`: permite el envío de la cookie desde dominios distintos al del backend —necesario cuando frontend y backend residen en servidores separados—, pero el estándar obliga a combinarlo con `Secure = true` [65]. Esta

configuración se complementa con una política CORS (Cross-Origin Resource Sharing) que restringe los dominios autorizados a realizar solicitudes a la API.

- **Expires:** establece la caducidad de la cookie en 8 horas mediante el valor definido en las variables de entorno.

En el entorno de producción previsto para el Instituto Finlay, donde ambos componentes compartirán el mismo dominio, la política CORS dejará de ser necesaria y **SameSite** podrá restringirse a **Lax**, lo que reforzará la seguridad al impedir el envío de la cookie en solicitudes cross-site.

Además, cada vez que el cliente utiliza un refresh token para obtener un nuevo JWT, el sistema aplica rotación: revoca el token anterior y emite uno nuevo. Si un atacante logra robar un refresh token, la rotación limita su utilidad a una única renovación, tras la cual el token legítimo queda invalidado y el usuario deberá iniciar sesión nuevamente.

Envío del token en cada solicitud El cliente recibe el JWT en el cuerpo de la respuesta de inicio de sesión y lo almacena en memoria. En cada solicitud subsiguiente, lo incluye en el encabezado **Authorization: Bearer <token>**, conforme al estándar definido en el RFC 6750 [66]. El refresh token, por su parte, viaja automáticamente en la cookie sin intervención del código frontend, lo que reduce la superficie de ataque al no exponerlo a lógica JavaScript.

Autorización basada en roles

Una vez que el middleware de autenticación valida el JWT y establece la identidad del usuario, el sistema delega el control de acceso en dos mecanismos complementarios: el atributo **[Authorize]** para restringir endpoints por rol, y un servicio de contexto de usuario para recuperar la identidad del usuario autenticado dentro de la capa de aplicación.

El atributo **[Authorize(Roles = "SectionResponsible")]** indica al middleware de autorización que solo los usuarios cuyo claim **Role** coincida con ese valor pueden ejecutar el método. Si el token no contiene el rol requerido, el middleware retorna automáticamente un código 403 (Forbidden) con un mensaje JSON, según la configuración establecida en el evento **OnForbidden** del JWT Bearer.

Para que la capa de aplicación pueda acceder a la identidad del usuario autenticado sin que el frontend la envíe manualmente en cada solicitud, el sistema implementa **HttpContextService**. Este servicio extrae el claim **NameIdentifier** del token JWT validado —disponible en el **HttpContext** de la solicitud— y lo expone mediante el método **GetUserId()**:

```
1 public class UserContextService : IUserContextService
2 {
3     private readonly IHttpContextAccessor _httpContextAccessor;
4
5     public UserContextService(IHttpContextAccessor
6     httpContextAccessor)
7     {
8         _httpContextAccessor = httpContextAccessor;
9     }
10
11     public Guid GetUserId()
12     {
13         var user = _httpContextAccessor.HttpContext?.User
14         ?? throw new UnauthorizedAccessException("No
15         authenticated user.");
16
17         var userId = user.FindFirst(ClaimTypes.NameIdentifier)?.
18         Value
19         ?? throw new UnauthorizedAccessException("User ID not
20         found in token.");
21
22         return Guid.Parse(userId);
23     }
24 }
```

Ejemplo de código 1.34: Servicio de contexto de usuario autenticado

El método `GetUserId()` obtiene el `HttpContext` de la solicitud actual mediante `IHttpContextAccessor` y recupera el claim `ClaimTypes.NameIdentifier` —cuyo valor se estableció durante la generación del JWT con `JwtRegisteredClaimNames.Sub`—. Este enfoque elimina la necesidad de que cada endpoint reciba explícitamente el identificador del usuario como parámetro, lo que reduce el riesgo de suplantación: el identificador no puede ser manipulado por el cliente porque proviene del token validado por el servidor.

La capa de aplicación consume este servicio para personalizar las consultas según el usuario autenticado. El siguiente fragmento muestra cómo un revisor médico obtiene únicamente los reportes que le fueron asignados, sin recibir como parámetro su propio identificador:

```
1 public async Task<PagedResultDto<ReportMedicalReviewerDto>>
2     GetReportAssignment(
3         PagedRequestDto paged,
4         ReportMedicalReviewerFilter filter)
5 {
6     var userId = _userContextService.GetUserId();
7
8     var medicalReviewer = await _unitOfWork
9     .GetRepository<MedicalReviewer>()
```

```
9         .FirstOrDefaultAsync(sr => sr.UserId == userId)
10         ?? throw new UnauthorizedAccessException("User is not a
    medical reviewer");
11
12     var reportIds = _unitOfWork
13         .GetRepository<MedicalReviewAssignment>()
14         .GetAllByItems(mra =>
15             mra.MedicalReviewerId == medicalReviewer.Id &&
16             mra.Status == ReviewAssignmentStatus.Pending)
17         .Select(a => a.AefiReportId)
18         .Distinct();
19
20     var query = _unitOfWork
21         .GetRepository<AefiReport>()
22         .GetAllByItems(r => reportIds.Contains(r.Id));
23
24     // Filtrado, paginación y proyección a DTO...
25 }
```

Ejemplo de código 1.35: Uso de `UserService` en la capa de aplicación

El método obtiene el identificador del usuario desde el token, lo utiliza para localizar al revisor médico asociado y filtra los reportes cuyas asignaciones coincidan con ese revisor. En ningún momento el frontend envía el identificador del usuario; este se deriva exclusivamente del token validado, lo que garantiza que un usuario autenticado solo pueda acceder a sus propios recursos.

Validación de datos y protección ante ataques automatizados

Los endpoints públicos del sistema —inicio de sesión, registro de reportes, consulta por número de notificación— están expuestos a ataques automatizados que pueden saturar la base de datos con reportes falsos o probar credenciales por fuerza bruta. Como primera barrera, el sistema exige que cada solicitud incluya un token que demuestre que proviene de un humano.

Entre los distintos tipos de CAPTCHA, el sistema opta por Friendly CAPTCHA, un servicio basado en prueba de trabajo (proof-of-work) que resuelve el desafío criptográfico en segundo plano sin interrumpir al usuario. La documentación oficial describe que este mecanismo exige al cliente resolver un cálculo criptográfico antes de enviar la solicitud, y que el servicio ajusta la dificultad del desafío en función del comportamiento del cliente para dificultar ataques automatizados [67].

La Figura 1.1 ilustra el flujo de verificación: un widget —un pequeño fragmento de código JavaScript que Friendly CAPTCHA proporciona para incrustar en los formularios del frontend— resuelve el desafío mientras el usuario completa los campos, adjunta un token a la solicitud y el controlador lo verifica antes de ejecutar el caso de uso.

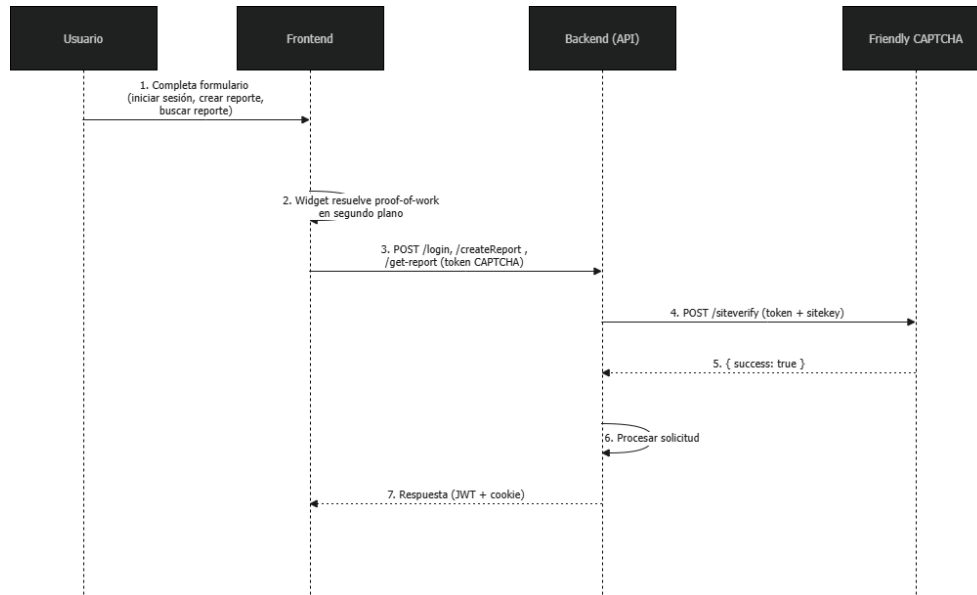


Figura 1.1: Flujo de verificación de Friendly CAPTCHA en endpoints públicos.

Limitación de velocidad

La limitación de velocidad (en inglés, *rate limiting*) restringe el número de solicitudes que un cliente puede realizar a la API en un período de tiempo determinado. Su propósito es proteger el sistema contra ataques de fuerza bruta, enumeración de recursos y denegación de servicio [68]. Además, evita que picos de tráfico legítimo o bucles accidentales en el frontend consuman todos los recursos del servidor y degraden la experiencia del resto de usuarios.

Los endpoints públicos del sistema ya cuentan con CAPTCHA como primera barrera, según se describió anteriormente. El rate limiting actúa como segunda barrera: si un atacante resuelve el CAPTCHA para cada intento, el límite de solicitudes por minuto hace inviable cualquier ataque masivo. Para los endpoints protegidos con JWT, el rate limiting constituye la defensa principal contra el abuso por parte de usuarios autenticados o tokens comprometidos.

Limitación por IP y por sesión El sistema aplica un limitador global que contabiliza todas las solicitudes de un mismo cliente, independientemente del endpoint al que se dirijan. Para clientes anónimos —como un reportante que envía un formulario público—, el límite se aplica por dirección IP: 100 solicitudes por minuto. Para usuarios autenticados, el límite se aplica por identificador de sesión extraído del JWT: 200 solicitudes por minuto. Esta diferenciación evita que usuarios legítimos que comparten red —como el personal de un mismo centro de salud— resulten bloqueados

por el comportamiento de un tercero, y otorga mayor margen a quienes ya superaron la barrera de autenticación. Una solicitud solo se procesa si el limitador global lo permite.

Políticas complementarias por tipo de operación Sobre esta base, el sistema añade políticas específicas que restringen aún más ciertos endpoints según la criticidad de la operación. Cada política define un límite de solicitudes, una ventana de tiempo y un comportamiento de encolamiento. La ventana fija restablece el contador al cumplirse el intervalo; la ventana deslizante distribuye el límite en segmentos para suavizar los picos de tráfico legítimo. El encolamiento permite retener las solicitudes excedentes durante un breve período antes de rechazarlas, lo que evita que un pico puntual de actividad legítima genere errores inmediatos al cliente [68].

- **Auth:** 3 solicitudes por minuto para endpoints de autenticación, sin encolamiento. Un usuario legítimo inicia sesión pocas veces al día; tres intentos por minuto bastan para la operación normal y hacen inviable un ataque de fuerza bruta incluso si el atacante resuelve el CAPTCHA en cada intento.
- **Command:** 5 solicitudes por minuto para operaciones de escritura estándar, como la creación de reportes ESAVI, con cola de hasta 5 solicitudes. Equivale a un reporte cada 12 segundos, ritmo suficiente para un centro de salud con múltiples reportantes simultáneos.
- **ReportDaily:** 30 solicitudes por día para la creación de reportes desde una misma IP, mediante ventana deslizante de 24 horas dividida en 24 segmentos. Cubre la actividad diaria de un centro de salud activo y acota el daño máximo de un ataque automatizado a 30 reportes falsos, los cuales pueden identificarse y eliminarse manualmente.
- **CommandDaily:** 20 solicitudes por día para operaciones administrativas como la asignación de reportes o la finalización de revisiones médicas. Refleja el ritmo real de trabajo de un jefe de vacunación municipal o médico evaluador.
- **Query:** 60 solicitudes por minuto para operaciones de lectura, con ventana deslizante dividida en 3 segmentos y cola de hasta 10 solicitudes. Las consultas no modifican el estado del sistema; una consulta por segundo permite una navegación fluida mientras dificulta la extracción masiva de datos.

El siguiente fragmento muestra la configuración de dos de estas políticas dentro de la clase `RateLimitingMiddleware`:

```

1 // Autenticación: muy restrictivo
2 options.AddFixedWindowLimiter("Auth", config =>
3 {
4     config.PermitLimit = 3;
5     config.Window = TimeSpan.FromMinutes(1);
6     config.QueueProcessingOrder = QueueProcessingOrder.OldestFirst;
7     config.QueueLimit = 0;
8 });
9
10 // Comandos estándar: restrictivo
11 options.AddFixedWindowLimiter("Command", config =>
12 {
13     config.PermitLimit = 5;
14     config.Window = TimeSpan.FromMinutes(1);
15     config.QueueProcessingOrder = QueueProcessingOrder.OldestFirst;
16     config.QueueLimit = 5;
17 });

```

Ejemplo de código 1.36: Configuración de políticas de rate limiting

Aplicación a endpoints El atributo `[EnableRateLimiting]` aplica las políticas sobre controladores completos o endpoints específicos. Un mismo endpoint puede combinar múltiples políticas cuando requiere protección en distintas ventanas de tiempo. Por ejemplo, el controlador de autenticación declara únicamente `Auth` a nivel de clase:

```

1 [ApiController]
2 [Route("api/[controller]")]
3 [EnableRateLimiting("Auth")]
4 public class AuthenticationController : ControllerBase { }

```

Ejemplo de código 1.37: Aplicación del rate limiting a controladores y endpoints

El endpoint de creación de reportes, en cambio, combina dos políticas: `Command` limita la tasa por minuto y `ReportDaily` impone un límite diario por IP. Ambas se evalúan en conjunto con el limitador global: la solicitud solo se procesa si las tres capas lo permiten.

```

1 [HttpPost]
2 [EnableRateLimiting("Command")]
3 [EnableRateLimiting("ReportDaily")]
4 public async Task<IActionResult> Create(CreateReportDto dto) { }

```

Ejemplo de código 1.38: Combinación de políticas en un mismo endpoint

Alcance de las defensas implementadas Aun con estas barreras, un atacante necesitaría resolver múltiples desafíos de proof-of-work y mantenerse por debajo de los umbrales de rate limiting simultáneamente. El costo computacional de esta estrategia la vuelve inviable para ataques masivos con los recursos actuales.

1.4. Implementación de la interfaz de usuario

El frontend del sistema se construye como una Single Page Application (SPA) en React con TypeScript. La aplicación consume la API REST del backend y ofrece una interfaz accesible tanto para la población que reporta eventos adversos como para el personal del Instituto Finlay que gestiona las revisiones.

1.4.1. Arquitectura y enrutamiento

La aplicación utiliza enrutamiento interno sin depender de librerías externas como React Router. Un componente principal mantiene el estado de la página actual y sincroniza la ruta del navegador con el componente correspondiente mediante un mapa de rutas. Esta decisión reduce el tamaño del bundle final y permite animaciones de transición entre páginas. El acceso a las páginas restringidas se protege mediante un contexto de autenticación que redirige al usuario a la página de inicio de sesión si no hay una sesión activa. Los contextos globales de autenticación y reportes envuelven toda la aplicación y comparten el estado sin necesidad de pasarlo manualmente entre componentes.

1.4.2. Gestión del estado y comunicación con la API

El estado global de autenticación reside en un contexto de React que expone los datos del usuario, el estado de la sesión y las funciones de inicio y cierre de sesión. Al cargar la aplicación, este contexto intenta restaurar la sesión mediante el endpoint de refresco de token, lo que evita que el usuario deba iniciar sesión nuevamente si su token de acceso expiró.

La comunicación con el backend se centraliza en una instancia de Axios configurada con interceptores para la gestión automática de tokens. El interceptor de solicitud inyecta el encabezado de autorización con el token de acceso; el interceptor de respuesta detecta los códigos 401, intenta renovar el token y reenvía la solicitud original. El refresco excluye los endpoints de autenticación para evitar bucles y preserva los encabezados originales en el reintento. Si la renovación falla, la sesión se cierra automáticamente y el usuario regresa a la página de inicio de sesión.

1.4.3. Almacenamiento del JWT en memoria

El token de acceso se mantiene exclusivamente en memoria, sin persistencia en almacenamiento local o de sesión. Esta decisión reduce el riesgo de robo por lecturas desde almacenamiento persistente, aunque no protege contra scripts inyectados que se ejecuten en la página. Por ello, el patrón de seguridad completo combina el token de acceso en memoria con un refresh token almacenado en una cookie HttpOnly, Secure

y con SameSite estricta, que el frontend no manipula y por tanto no es vulnerable a ataques de cross-site scripting.

Esta estrategia conlleva un riesgo: el token de acceso se pierde al recargar la página. Para mitigarlo, el contexto de autenticación invoca el endpoint de refresco al cargar la aplicación. Si el refresco tiene éxito, el nuevo token se almacena en memoria y la sesión se restaura sin interrupción para el usuario.

1.4.4. Validación en tiempo real

Los formularios del sistema aplican validación en tiempo real mediante las librerías `react-hook-form` y `zod`. La primera reduce el número de renderizaciones al no almacenar el estado de cada campo individualmente, lo que mejora el rendimiento en formularios extensos como el de registro de reportes. La segunda define esquemas declarativos que especifican el formato esperado de cada campo —correo electrónico válido, longitud mínima de contraseña, coincidencia entre campos— y los mensajes de error correspondientes. La integración de ambas permite que el usuario reciba retroalimentación inmediata mientras escribe, sin esperar a enviar el formulario, lo que reduce la cantidad de solicitudes fallidas al backend y mejora la experiencia de uso.

1.4.5. Construcción de interfaces de usuario y diseño responsivo

La interfaz se compone de componentes reutilizables contruidos con Tailwind CSS y Radix UI. Tailwind CSS proporciona utilidades de diseño que permiten crear interfaces responsivas sin escribir hojas de estilo personalizadas: las cuadrículas reorganizan sus columnas según el ancho de la pantalla, la navegación alterna entre un menú de escritorio y un menú lateral en dispositivos móviles, y los formularios mantienen un aspecto adecuado en cualquier tamaño de pantalla. Radix UI aporta componentes accesibles —diálogos, selectores, pestañas— que garantizan la compatibilidad con lectores de pantalla y navegación por teclado, en cumplimiento de los requisitos de accesibilidad.

Los gráficos del panel de control utilizan `recharts`, una librería de visualización de datos para React. Cada gráfico se envuelve en un contenedor responsivo que adapta su tamaño al ancho disponible, y los datos se preprocesan en el backend para reducir el volumen de información transferida al cliente. Los componentes visuales emplean memoización para evitar renderizaciones innecesarias y mantener un rendimiento fluido durante la navegación.

1.4.6. Manejo de autenticación en el cliente

Al iniciar sesión, el cliente decodifica el JWT para extraer el rol del usuario y mostrar únicamente las rutas y opciones correspondientes a ese perfil. Esta diferenciación garantiza que cada perfil acceda exclusivamente a las funcionalidades que le corresponden, en cumplimiento de los requisitos de autorización definidos en el Capítulo ??.

1.5. Despliegue de la solución

El sistema se despliega en dos entornos diferenciados. Para desarrollo local, un archivo `docker-compose.yml` levanta los servicios de RabbitMQ y la API, mientras que la base de datos MySQL se ejecuta en el servidor local. Para demostración y pruebas, la API se despliega en Railway, una plataforma como servicio que automatiza la construcción y publicación desde el repositorio de GitHub.

La configuración se centraliza en un `Dockerfile` multietapa⁵ que compila la aplicación en una imagen de SDK⁶ y la publica en una imagen de runtime⁷, y en variables de entorno que inyectan las credenciales de base de datos, las claves de JWT y las API keys de los servicios externos sin exponerlas en el código fuente.

La migración a servidores físicos del Instituto Finlay, prevista para la etapa de producción, mantendrá la misma configuración de contenedores y variables de entorno, lo que garantiza que el entorno de producción sea idéntico al de desarrollo.

1.6. Conclusiones del capítulo

En este capítulo se presentaron los principales detalles relacionados con la implementación de la plataforma desarrollada, describiendo las tecnologías empleadas, la organización arquitectónica de la solución y los mecanismos utilizados en el backend y el frontend. Además, se abordaron aspectos asociados a la seguridad, la integración entre componentes y el despliegue del sistema, evidenciando la materialización práctica de la propuesta presentada en el capítulo anterior.

⁵Multietapa: técnica que divide la construcción en fases con imágenes base distintas, reduciendo el tamaño final.

⁶SDK: imagen con compilador y herramientas de desarrollo.

⁷Runtime: imagen solo con las bibliotecas para ejecutar la aplicación.

Capítulo 2

Validación de los resultados

Apéndice A

Resultados de farmacovigilancia pasiva en Cuba y las Américas

A.1. Reporte de RAM notificadas por farmacovigilancia pasiva en Cuba

En el Anexo A.1 se presenta una figura con la evolución de las notificaciones de reacciones adversas a medicamentos (RAM) en Cuba entre 2014 y 2020. Estos datos corresponden al sistema de **farmacovigilancia pasiva**, basado en la notificación espontánea de sospechas de RAM por parte del personal sanitario.

Año	Número de reportes RAM	Tasa de reporte por millón de habitantes
2014	20 380	1812
2015	20 611	1833
2016	18 977	1688
2017	17 070	1519
2018	14 736	1312
2019	15 982	1425
2020	13 025	1163

Fuente: Base de datos nacional de farmacovigilancia

Figura A.1: Notificaciones de reacción adversa a medicamentos (RAM) declaradas mediante farmacovigilancia pasiva en Cuba (2014-2020). Fuente: CECMED, Informe Anual 2020 [69].

A.2. Reporte según grupo farmacológico

En el Anexo A.2 se incluye una figura que muestra los reportes de reacción adversa a medicamentos (RAM) según el grupo farmacológico, a partir de la farmacovigilancia pasiva del año 2020.

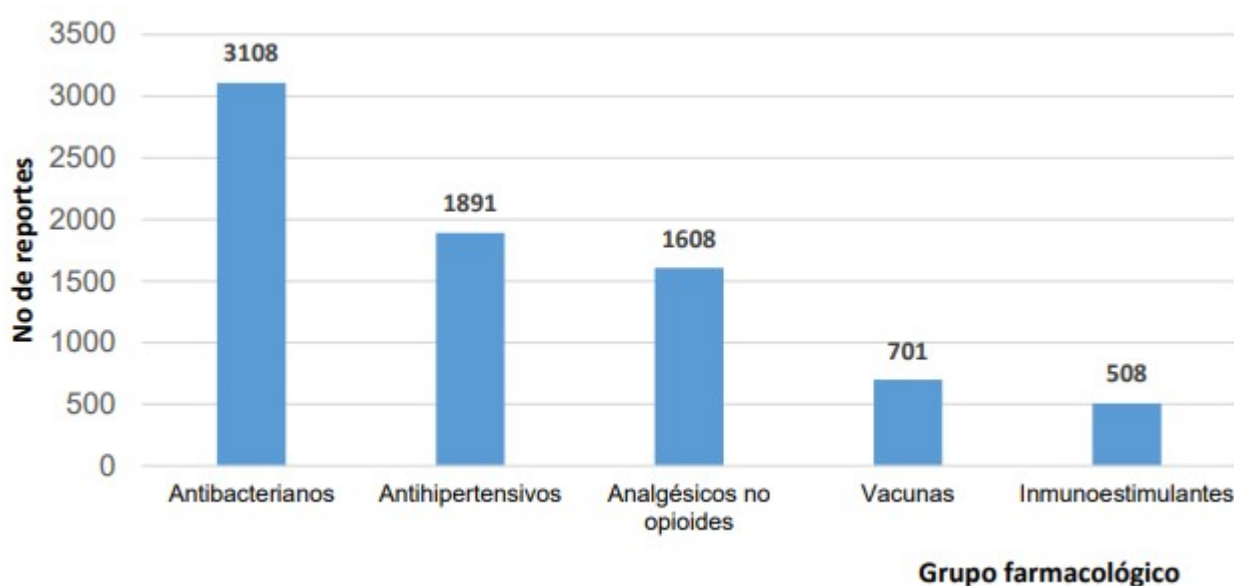


Figura A.2: Reporte de reacción adversa a medicamentos (RAM) según grupo farmacológico. Farmacovigilancia pasiva, 2020. Adaptado de [69]

A.3. Notificaciones de ESAVI por países de la región de las Américas

En el Anexo A.3 se incluye una figura que presenta el número de ESAVI notificados por los países de la región de las Américas.



Figura A.3: Número de ESAMI notificados por los países de la región de las Américas. Adaptado de [70]

A.4. Estructura del Sistema de Vigilancia de Medicamentos en Cuba

En el Anexo A.4 se incluye una figura que presenta la organización del sistema cubano de vigilancia de medicamentos.

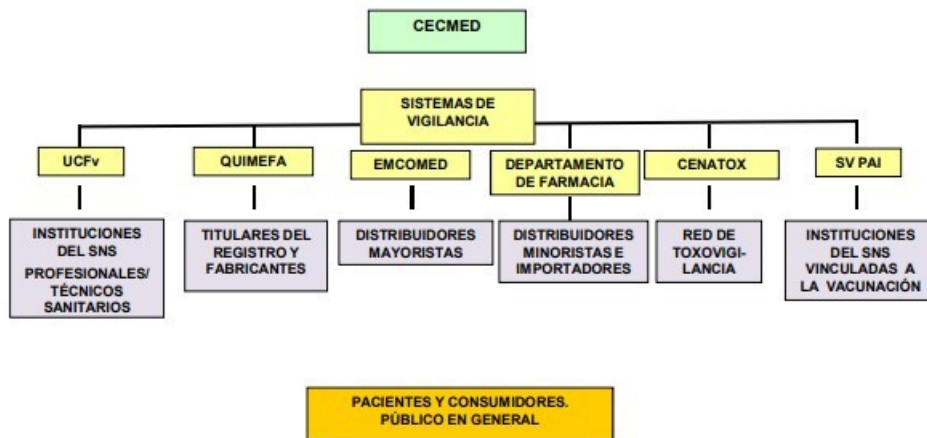


Figura A.4: Estructura del Sistema de Vigilancia de Medicamentos en Cuba. Adaptado de: [71]

El sistema de vigilancia de medicamentos en Cuba se estructura bajo la rectoría del CECMED, autoridad reguladora nacional encargada de la evaluación, control y toma de decisiones sobre la seguridad de los medicamentos a partir de la información generada en el sistema.

Para el cumplimiento de estas funciones, se articula una red de vigilancia integrada por diversos componentes que interactúan de manera coordinada (ver Figura A.4) [71]:

- **UCNFV (MINSAP):** Cohesiona la Red de Farmacoepidemiología en los 169 municipios del país y actúa como el canal central para la notificación espontánea de profesionales y pacientes, siendo las vacunas la excepción gestionada de forma especializada por el PAI [71, 72].
- **Fabricantes y Titulares de Registro (QUIMEFA):** Involucra a las 24 empresas farmacéuticas nacionales, donde cada una cuenta con personal designado para la vigilancia y reporta directamente sobre la seguridad y calidad de sus productos [71].
- **Distribución Mayorista (EMCOMED):** Coordina la vigilancia en las 24 droguerías del país, detectando problemas de calidad durante el almacenamiento y transportación [71].
- **Distribución Minorista (Departamento de Farmacia):** Supervisa la vigilancia en las más de 2,000 farmacias comunitarias y hospitalarias del Sistema Nacional de Salud [71].
- **CENATOX:** Lidera la red de toxicovigilancia, supervisando las intoxicaciones y reacciones adversas de naturaleza tóxica [71].
- **SV PAI:** Realiza la vigilancia especializada de vacunas mediante el monitoreo de los ESAVI, bajo la supervisión del Viceministerio de Higiene y Epidemiología [71].

Toda la información captada por estos efectores fluye hacia el Observatorio de Vigilancia Postcomercialización del CECMED, donde se evalúa la relación beneficio-riesgo para emitir alertas farmacológicas, retirar lotes o modificar registros sanitarios en tiempo real [71].

Apéndice B

Modelos oficiales de notificación en Cuba

B.1. Modelo 33-36-02

En el Anexo B.1 se incluye el **Modelo 33-36-02**, el formulario oficial utilizado por los profesionales de la salud en Cuba para la notificación de sospechas de reacciones adversas a medicamentos dentro del sistema nacional de farmacovigilancia.

Modelo 33-36-02 MINISTERIO DE SALUD PÚBLICA Farmacia	NOTIFICACIÓN DE SOSPECHA DE REACCIÓN ADVERSA A MEDICAMENTOS POR EL PROFESIONAL SANITARIO										Pág. 1 de 1				
											Mes				
											Año				
Unidad que reporta:				Provincia:				Municipio:							
PACIENTE: Nombre y apellidos: _____ Edad: _____ Sexo: F ☐ M ☐ Color de la piel: (kg) ☐ ☐ ☐ ☐ Peso: _____															
Nombre del que reporta:				Médico ☐ Lic. Farmacia ☐ Téc. Farmacia ☐ Lic. Enfermería ☐ Enfermera ☐ Estomatólogo ☐ Otro ☐											
Medicamentos tomados hasta 3 meses antes de la RAM. Marcar con una cruz los sospechosos		Lote	Vía de administración	Dosis diaria	TRATAMIENTO Inicio Fin Día Mes Año Día Mes Año						Motivo de Prescripción				
Fabricante:				Patologías que presenta el paciente:											
REACCIONES. Enumérense por separado				Fecha de inicio			Fecha de término			Requirió ingreso o prolongó su estadía hospitalaria		Atención de urgencia		Puso en peligro su vida	
				Día	Mes	Año	Día	Mes	Año	SI	NO	SI	NO	SI	NO
DESENLACE										SI	NO				
Recuperado		☐	¿Se ha suspendido la medicación?								Fecha de notificación:				
No recuperado		☐	¿Ha mejorado al suspenderla?												
Recuperado con secuela		☐	¿Se administró nuevamente este medicamento?												
Mortal		☐	¿Si se administró nuevamente, hubo recurrencia de síntomas?								Día	Mes	Año		
Observaciones adicionales:															

Figura B.1: Modelo 33-36-02: Notificación de sospechas de Reacciones Adversas a Medicamentos por el Profesional Sanitario. Adaptado de [72]

Notificación de sospechas de Reacciones Adversas a Medicamentos

Informe Confidencial

1. Por favor notifique todas las reacciones a los fármacos recientemente introducidos en el mercado y todas las reacciones moderadas, severas, raras y mortales a todos los medicamentos, productos estomatológicos, quirúrgicos, DIU, suturas y lentes de contacto.
2. Indique el/los medicamentos a que se atribuyen los efectos nocivos causados al paciente (en caso de existir más de uno marque con un asterisco el nombre del más sospechoso). Indique además todos los medicamentos administrados los tres meses anteriores.
3. No deje de notificar reacciones adversas raras por considerar que no estén relacionadas con el medicamento, estas pueden ser las más importantes.
4. En caso de anomalías congénitas, aunque estas fueran menores, indique todos los medicamentos tomados por la madre durante la gestación. En el acápite patologías que presenta el paciente poner las patologías maternas crónicas y las padecidas durante la gestación, sin olvidar las etiologías infecciosas.
5. Debe poner en observaciones especiales cualquier dato de interés que crea importante incluir, incluyendo procederes que hubo de indicársele al paciente para el tratamiento de la reacción, hábitos tóxicos, resultados de exámenes complementarios, etc.
6. Trate siempre de llenar la planilla completa, sin omitir nada, pero NO DEJE DE NOTIFICAR PORQUE LE FALTE ALGUN DATO.

ESTA PLANILLA DEBE ENVIARSE A LA FARMACIA PRINCIPAL MUNICIPAL QUE LA HARA LLEGAR A LA UNIDAD PROVINCIAL DE FARMACOVIGILANCIA UBICADA EN LA VICEDIRECCION DE MEDICAMENTOS DE LA DIRECCION PROVINCIAL DE SALUD.

Cualquier consulta o ampliación de la información contactar con:

Departamento de Farmacoepidemiología

Unidad Coordinadora Nacional de Farmacovigilancia

MINSAP 23 y N 7mo piso Vedado

Teléfono: 839-63-60

Email: giset@msp.sld.cu, ismary@msp.sld.cu

Figura B.2: Instructivo en el dorso del modelo 33-36-02: Notificación de sospechas de Reacciones Adversas a Medicamentos por el Profesional Sanitario, Adaptado de [72]

B.2. Modelo 33-36-03

En el Anexo B.2 se incluye el **Modelo 33-36-03**, el formulario oficial utilizado por los pacientes en Cuba para la notificación de sospechas de reacciones adversas a medicamentos dentro del sistema nacional de farmacovigilancia.

Unidad que reporta:				Provincia				Municipio							
PACIENTE:															
Nombre y Apellidos _____															
Edad: _____				Sexo: F ☐ M ☐		Color de la Piel: Pst ☐ kg ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐									
Nombre del que reporta: _____				Médico ☐ Lic. Farmacia ☐ Téc. Farmacia ☐ Lic. Enfermería ☐ Enfermera ☐ Estomatólogo ☐ Otro ☐											
Medicamentos tomados hasta 3 meses antes de la RAH. Marcar con una cruz los sospechosos		Lote	Vía de Administración	Dosis diaria	TRATAMIENTO						Motivo de Prescripción				
					Inicio Fin Día Mes Año Día Mes Año										
Fabricante: _____				Patologías que presenta el paciente _____											
REACCIONES. Enumérense por separado				Fecha de Inicio			Fecha de Término			Requirió ingreso o prolongó su estadía hospitalaria		Atención de Urgencia		Puso en peligro su vida	
				Día	Mes	Año	Día	Mes	Año	SI	NO	SI	NO	SI	NO

	SI	NO
¿Se ha suspendido la medicación?		
¿Ha mejorado al suspenderla?		
¿Se administró nuevamente este medicamento?		
¿Si se administró nuevamente, hubo recurrencia de síntomas?		

DESENLACE	
Recuperado	☐
No recuperado	☐
Recuperado con secuela	☐
Mortal	☐

Observaciones adicionales:

INSTRUCTIVO AL DORSO

Fecha de Notificación:

Día	Mes	Año

Figura B.3: Modelo 33-36-03: Notificación de sospechas de Reacciones Adversas a Medicamentos por el Paciente. Adaptado de [72]

Notificación de sospechas de Reacciones Adversas a Medicamentos
Informe Confidencial

1. Un efecto no deseado producido por medicamentos es un efecto que produce una incomodidad o daño al paciente, que ocurre después de consumir uno o más medicamentos siendo utilizados éstos a la dosis correcta para prevenir o tratar una enfermedad.
2. Por favor notifique todos los efectos no deseados a cualquier medicamento, productos estomatológicos, quirúrgicos, dispositivos intrauterinos, suturas y lentes de contacto.
3. Indique el/los medicamentos a que se atribuyen los efectos no deseados causados al paciente (en caso de existir más de uno marque con una cruz el nombre del más sospechoso). Indique además todos los medicamentos administrados los tres meses anteriores poniendo uno en cada línea.
4. En la columna para qué se lo indicó el médico ponga el motivo en la línea correspondiente a cada medicamento.
5. Cuando el que notifica es la persona que sufrió el efecto no deseado marque con una cruz la casilla Paciente, si se refiere a alguna otra persona, pues marca Persona que reporta.
6. La casilla Cantidad al día se refiere a la cantidad de medicamento (tableta, cucharada, inyección etc.) que ingirió en un día.
7. Los efectos no deseados deben ser escritos por separado en cada línea, poniendo en primer lugar si es posible, el que más molestias le pasó.
8. La casilla tiempo pasado entre el consumo del medicamento e inicio del efecto no deseado se refiere al tiempo transcurrido desde que comenzó a tomar el medicamento y comienza el efecto adverso.
9. No deje de notificar efectos no deseados por considerar que no estén relacionadas con el medicamento, estos pueden ser los más importantes.
10. Añada cualquier dato de interés que crea importante incluir en la casilla que pregunta si quiere decir algo más. Si desea poner el lote del medicamento o el fabricante puede ponerlo en esta casilla.
11. Trate siempre de llenar la planilla completa, sin omitir nada, pero **NO DEJE DE NOTIFICAR PORQUE LE FALTE ALGUN DATO.**

Figura B.4: Instructivo en el dorso del modelo 33-36-03: Notificación de sospechas de Reacciones Adversas a Medicamentos por el Paciente. Adaptado de [72]

B.3. Modelo 84-30-2

En el Anexo B.3 se incluye el **Modelo 84-30-2**, el formulario oficial utilizado por los profesionales de la salud en Cuba para la notificación de sospechas de eventos supuestamente atribuibles a la vacunación o inmunización (ESAVI) dentro del sistema nacional de farmacovigilancia.

Apéndice C

Base de Datos Central de Farmacovigilancia (FarmaVigiC)

C.1. Campos de la base de datos central de farmacovigilancia (FarmaVigiC)

En el Anexo C.1 se describen los campos de la base de datos central de farmacovigilancia (FarmaVigiC) utilizada en Cuba.

Tabla C.1: Campos de la base de datos central de farmacovigilancia (FarmaVigiC).
Elaboración propia a partir de [72] (p. 42).

Grupos campos	de	Variables o campos
Paciente		Nombre y apellidos Edad Sexo Raza Antecedentes patológicos personales Requirió ingreso Requirió atención de urgencia Requirió reposo Fue baja laboral o escolar Peligró su vida

Grupos de campos	Variables o campos
Notificador	Unidad que reporta Provincia Municipio Nombre y apellidos Especialidad
Medicamentos	Medicamento sospechoso Presentación cantidad Presentación unidad Forma farmacéutica Fabricante Diluyente Lote Vía de administración Dosis cantidad Dosis unidad Dosis frecuencia Fecha de inicio del tratamiento Fecha fin del tratamiento Continuación Motivo de prescripción Otro fármaco 1 Otro fármaco 2 Otro fármaco 3 Otro fármaco 4 Otro fármaco 5 Otro fármaco 6 Vía, dosis, lote de otros fármacos Fecha de administración de otros fármacos

Grupos de campos	Variables o campos
Reacciones adversas	Reacción principal Otra RAM 1 Otra RAM 2 Otra RAM 3 Otra RAM 4 Otra RAM 5 Otra RAM 6 Fecha de inicio RAM Fecha fin de RAM Continuación
Clasificación	Grupo de edades Sexo Nivel de atención Especialidad Grupo farmacológico Sistema de órgano afectado Severidad Causalidad Importancia Frecuencia
Desenlace	Suspendió la medicación Supresión positiva Re exposición Recurrencia de síntomas Desenlace Recuperado No recuperado Recuperado con secuela
Organización e información adicional	Mes de entrada a la base Secuencia temporal Observaciones Comentarios Resultados de necropsia Fecha de notificación
Evaluación de la calidad	E1 (evalúa al notificador) E2 (evalúa al revisor) E3 (evalúa calidad del llenado de la base)

- A nivel municipal se trabajan 71 campos
- A nivel provincial se trabajan 72 campos
- A nivel nacional se trabajan 75 campos

Apéndice D

Formularios de referencia

D.1. Formulario VAERS - Octubre 2025

En el Anexo D.1 se incluye el formulario oficial del Vaccine Adverse Event Reporting System (VAERS) correspondiente a octubre de 2025. Este formulario constituye el instrumento estándar utilizado para la notificación de eventos adversos posteriores a la vacunación en los Estados Unidos.

VAERS Vaccine Adverse Event Reporting System www.vaers.hhs.gov		Adverse events are possible reactions or problems that occur during or after vaccination. Items 2, 3, 4, 5, 6, 17, 18 and 21 are ESSENTIAL and should be completed. Patient identity is kept confidential. <u>Instructions</u> are provided on the last two pages.																														
INFORMATION ABOUT THE PATIENT WHO RECEIVED THE VACCINE (Use Continuation Page if needed)																																
1. Patient name: (first) _____ (last) _____ Street address: _____ City: _____ State: _____ County: _____ ZIP code: _____ Phone: () _____ Email: _____ 2. Date of birth: (mm/dd/yyyy) _____ 3. Sex: ☐ Male ☐ Female 4. Date and time of vaccination: (mm/dd/yyyy) _____ Time: hh:mm _____ 5. Date and time adverse event started: (mm/dd/yyyy) _____ Time: hh:mm _____ 6. Age at vaccination: Years _____ Months _____ 7. Today's date: (mm/dd/yyyy) _____ 8. Pregnant at time of vaccination?: ☐ Yes ☐ No ☐ Unknown (If yes, describe the event, any pregnancy complications, and estimated due date if known in item 18)	9. Prescriptions, over-the-counter medications, dietary supplements, or herbal remedies being taken at the time of vaccination: _____ 10. Allergies to medications, food, or other products: _____ 11. Other illnesses at the time of vaccination and up to one month prior: _____ 12. Chronic or long-standing health conditions: _____																															
INFORMATION ABOUT THE PERSON COMPLETING THIS FORM																																
13. Form completed by: (name) _____ Relation to patient: ☐ Healthcare professional/staff ☐ Patient (yourself) ☐ Parent/guardian/caregiver ☐ Other: _____ Street address: _____ ☐ Check if same as item 1 City: _____ State: _____ ZIP code: _____ Phone: () _____ Email: _____ 14. Best doctor/healthcare professional to contact about the adverse event: Name: _____ Phone: () _____ Ext: _____																																
INFORMATION ABOUT THE FACILITY WHERE VACCINE WAS GIVEN																																
15. Facility/clinic name: _____ Fax: () _____ Street address: _____ ☐ Check if same as item 13 City: _____ State: _____ ZIP code: _____ Phone: () _____ 16. Type of facility: (Check one) ☐ Doctor's office, urgent care, or hospital ☐ Pharmacy or store ☐ Workplace clinic ☐ Public health clinic ☐ Nursing home or senior living facility ☐ School or student health clinic ☐ Home ☐ Other: _____ ☐ Unknown																																
WHICH VACCINES WERE GIVEN? WHAT HAPPENED TO THE PATIENT?																																
17. Enter all vaccines given on the date listed in item 4: (Route is HOW vaccine was given, Body site is WHERE vaccine was given) <table border="1" style="width: 100%; border-collapse: collapse; margin-top: 5px;"> <thead> <tr> <th style="width: 30%;">Vaccine (type and brand name)</th> <th style="width: 20%;">Manufacturer</th> <th style="width: 15%;">Lot number</th> <th style="width: 10%;">Route</th> <th style="width: 10%;">Body site</th> <th style="width: 15%;">Dose number in series</th> </tr> </thead> <tbody> <tr><td>select</td><td>▼</td><td>▼</td><td>select</td><td>▼</td><td>select</td></tr> <tr><td>select</td><td>▼</td><td>▼</td><td>select</td><td>▼</td><td>select</td></tr> <tr><td>select</td><td>▼</td><td>▼</td><td>select</td><td>▼</td><td>select</td></tr> <tr><td>select</td><td>▼</td><td>▼</td><td>select</td><td>▼</td><td>select</td></tr> </tbody> </table>			Vaccine (type and brand name)	Manufacturer	Lot number	Route	Body site	Dose number in series	select	▼	▼	select	▼	select	select	▼	▼	select	▼	select	select	▼	▼	select	▼	select	select	▼	▼	select	▼	select
Vaccine (type and brand name)	Manufacturer	Lot number	Route	Body site	Dose number in series																											
select	▼	▼	select	▼	select																											
select	▼	▼	select	▼	select																											
select	▼	▼	select	▼	select																											
select	▼	▼	select	▼	select																											
18. Describe the adverse event(s), treatment, and outcome(s), if any: (symptoms, signs, time course, etc.) Use Continuation Page if needed																																
21. Result or outcome of adverse event(s): (Check all that apply) ☐ Doctor or other healthcare professional office/clinic visit ☐ Emergency room/department or urgent care ☐ Hospitalization: Number of days (if known) _____ Hospital name: _____ City: _____ State: _____ ☐ Prolongation of existing hospitalization (vaccine received during existing hospitalization) ☐ Life threatening illness (immediate risk of death from the event) ☐ Disability or permanent damage ☐ Patient died - Date of death: (mm/dd/yyyy) _____ ☐ Congenital anomaly or birth defect ☐ None of the above																																
19. Medical tests and laboratory results related to the adverse event(s): (include dates) Use Continuation Page if needed																																
20. Has the patient recovered from the adverse event(s)?: ☐ Yes ☐ No ☐ Unknown																																
ADDITIONAL INFORMATION																																
22. Any other vaccines received within one month prior to the date listed in item 4: <table border="1" style="width: 100%; border-collapse: collapse; margin-top: 5px;"> <thead> <tr> <th style="width: 30%;">Vaccine (type and brand name)</th> <th style="width: 20%;">Manufacturer</th> <th style="width: 15%;">Lot number</th> <th style="width: 10%;">Route</th> <th style="width: 10%;">Body site</th> <th style="width: 15%;">Dose number in series</th> <th style="width: 10%;">Date Given</th> </tr> </thead> <tbody> <tr><td>select</td><td>▼</td><td>▼</td><td>select</td><td>▼</td><td>select</td><td></td></tr> <tr><td>select</td><td>▼</td><td>▼</td><td>select</td><td>▼</td><td>select</td><td></td></tr> </tbody> </table>			Vaccine (type and brand name)	Manufacturer	Lot number	Route	Body site	Dose number in series	Date Given	select	▼	▼	select	▼	select		select	▼	▼	select	▼	select										
Vaccine (type and brand name)	Manufacturer	Lot number	Route	Body site	Dose number in series	Date Given																										
select	▼	▼	select	▼	select																											
select	▼	▼	select	▼	select																											
23. Has the patient ever had an adverse event following any previous vaccine?: (If yes, describe adverse event, patient age at vaccination, vaccination dates, vaccine type, and brand name) ☐ Yes _____ ☐ No ☐ Unknown																																
24. Patient's race: ☐ American Indian or Alaska Native ☐ Asian ☐ Black or African American ☐ Native Hawaiian or Other Pacific Islander ☐ White ☐ Unknown ☐ Other: _____																																
25. Patient's ethnicity: ☐ Hispanic or Latino ☐ Not Hispanic or Latino ☐ Unknown																																
26. Immuniz. proj. report number: (Health Dept use only) _____																																
COMPLETE ONLY FOR U.S. MILITARY/DEPARTMENT OF DEFENSE (DoD) RELATED REPORTS																																
27. Status at vaccination: ☐ Active duty ☐ Reserve ☐ National Guard ☐ Beneficiary ☐ Other: _____																																
28. Vaccinated at Military/DoD site: ☐ Yes ☐ No																																

FORM FDA VAERS 2.0 (10/25)

SAVE

Figura D.1: Formulario VAERS - Octubre 2025

D.2. Formulario v-safe - Enero 2021

En el Anexo D.2 se incluyen capturas del sistema v-safe del CDC correspondientes a enero de 2021. Este sistema constituye una herramienta de seguimiento activo utilizada para monitorizar síntomas y eventos adversos posteriores a la vacunación.

Let's start today's health check-in.

How are you feeling today? *



Good



Fair



Poor

Next >

Your information in v-safe is protected by administrative, technical, and physical measures that safeguard the confidentiality, integrity, and privacy of personal information. To the extent v-safe uses existing information systems managed by CDC, FDA, and other federal agencies, the systems employ strict security measures appropriate for the level of sensitivity of the data. [Learn more about v-safe.](#)

Figura D.2: Estado general

Fever Check

Have you had a fever or felt feverish today? *

☐ Yes
☐ No

< Previous

Next >

Your information in v-safe is protected by administrative, technical, and physical measures that safeguard the confidentiality, integrity, and privacy of personal information. To the extent v-safe uses existing information systems managed by CDC, FDA, and other federal agencies, the systems employ strict security measures appropriate for the level of sensitivity of the data. [Learn more about v-safe.](#)




Figura D.3: Verificación de fiebre

Symptom Check

Have you had any of these symptoms where you got the shot (injection site)? *

Select all that apply.

☐ Pain
☐ Redness
☐ Swelling
☐ Itching
☐ None

Have you experienced any of these symptoms today? *

Select all that apply.

☐ Chills
☐ Headache
☐ Joint pains
☐ Muscle or body aches
☐ Fatigue or tiredness
☐ Nausea
☐ Vomiting
☐ Diarrhea
☐ Abdominal pain
☐ Rash, not including the immediate area around the injection site
☐ None

Any other symptoms or health conditions you want to report

< Previous

Next >

Your information in v-safe is protected by administrative, technical, and physical measures that safeguard the confidentiality, integrity, and privacy of personal information. To the extent v-safe uses existing information systems managed by CDC, FDA, and other federal agencies, the systems employ strict security measures appropriate for the level of sensitivity of the data. [Learn more about v-safe.](#)

Figura D.4: Verificación de síntomas

Health Impact

Did any of the symptoms or health conditions you reported today cause you to: *

Select all that apply.

☐ Be unable to work
☐ Be unable to do your normal daily activities
☐ Get care from a doctor or other healthcare professional
☐ None of the above

< Previous

Submit

Your information in v-safe is protected by administrative, technical, and physical measures that safeguard the confidentiality, integrity, and privacy of personal information. To the extent v-safe uses existing information systems managed by CDC, FDA, and other federal agencies, the systems employ strict security measures appropriate for the level of sensitivity of the data. [Learn more about v-safe.](#)




Figura D.5: Impacto en la salud

Figura D.6: Capturas del sistema v-safe para el seguimiento de síntomas posteriores a la vacunación

Apéndice E

Entidades de infraestructura

E.1. RefreshToken

La entidad `RefreshToken` permite la renovación de sesiones sin que el usuario deba iniciar sesión nuevamente. Cuando un usuario se autentica, el sistema genera un token de acceso JWT de corta duración y un token de actualización que persiste en esta tabla. Al expirar el token de acceso, el cliente presenta el token de actualización y obtiene uno nuevo.

```
1 public class RefreshToken : GuidEntity
2 {
3     public string Token { get; set; } = null!;
4     public DateTime ExpiresAt { get; set; }
5     public bool Revoked { get; set; }
6     public Guid UserId { get; set; }
7     public User User { get; set; } = null!;
8 }
```

Ejemplo de código E.1: Entidad `RefreshToken`

`Token` almacena el valor del token de actualización. `ExpiresAt` define su vencimiento. `Revoked` permite invalidarlo antes de esa fecha. `UserId` y `User` asocian cada token con su usuario.

E.2. AuditLog

La entidad `AuditLog` registra las operaciones realizadas sobre el sistema y constituye la base para las funcionalidades de auditoría y trazabilidad descritas en la sección 1.3 del Capítulo 4. Contempla tanto usuarios autenticados como reporteros anónimos.

Tabla E.1: Campos de la entidad AuditLog

Campo	Tipo	Descripción
UserId	Guid?	Usuario autenticado (nulo si es anónimo)
UserEmail	string?	Correo del usuario autenticado
ReporterName	string?	Nombre del reportero anónimo
ReporterEmail	string?	Correo del reportero anónimo
Action	string	Operación: Create, Update, Delete, View
EntityName	string	Entidad afectada (ej. AefiReport)
EntityId	Guid?	Identificador del registro afectado
Details	string?	Descripción legible de la acción
OldValues	string?	Datos antes del cambio (JSON)
NewValues	string?	Datos después del cambio (JSON)
IpAddress	string?	Dirección IP de origen
CreatedAt	DateTime	Fecha y hora de la operación

Glosario

DRY (Don't Repeat Yourself) Principio formulado por Andrew Hunt y David Thomas que establece que cada pieza de conocimiento debe tener una representación única, inequívoca y autoritativa dentro del sistema. La duplicación de lógica aumenta el riesgo de inconsistencias cuando se requiere un cambio [74].

16

ESAVI Evento Supuestamente Atribuible a la Vacunación o Inmunización. Cualquier evento adverso que se sospecha relacionado con la administración de una vacuna. 5, 8, 36, 75–77

notificación espontánea Sistema de reporte voluntario en el que profesionales de la salud, pacientes o cualquier persona pueden informar sobre sospechas de reacciones adversas a medicamentos sin necesidad de una solicitud formal. 77

reacción adversa a medicamentos (RAM) Según la OMS, reacción nociva y no deseada que se presenta tras la administración de un medicamento, a dosis utilizadas habitualmente en la especie humana, para prevenir, diagnosticar o tratar una enfermedad, o para modificar cualquier función biológica". Nótese que esta definición implica una relación de causalidad entre la administración del medicamento y la aparición de la reacción . 74, 75

SOLID Acrónimo de cinco principios de diseño orientado a objetos:

- **Responsabilidad Única** (una clase debe tener una sola razón para cambiar),
- **Abierto/Cerrado** (abierto a extensión, cerrado a modificación),
- **Sustitución de Liskov** (los objetos de una subclase deben poder sustituir a los de su clase base sin alterar el sistema),
- **Segregación de Interfaces** (interfaces pequeñas y específicas en lugar de una interfaz grande y genérica),

- **Inversión de Dependencias** (depender de abstracciones, no de implementaciones concretas).

Estos principios fueron recopilados por Robert C. Martin [75] y el acrónimo fue acuñado por Michael Feathers [76]. 16

tóxica Relacionada con sustancias dañinas, venenosas o nocivas para la salud de los organismos vivos. 77

Siglas

CECMED Centro para el Control Estatal de Medicamentos, Equipos y Dispositivos Médicos. 74, 76, 77

CENATOX Centro Nacional de Toxicovigilancia. 77

IFV Instituto Finlay de Vacunas. 9

MINSAP Ministerio de Salud Pública de Cuba. 21, 77

PAI Programa de Atención Integral. 77

UCNFV Unidad Coordinadora Nacional de Farmacovigilancia. 77

Bibliografía

- [1] Shari Lawrence Pfleeger y Joanne M. Atlee. *Software Engineering: Theory and Practice*. 4th. Pearson, 2015.
- [2] Ian Sommerville. *Software Engineering*. 10th. Pearson Education, 2015.
- [3] Winston W. Royce. «Managing the Development of Large Software Systems». En: *Proceedings of the 9th International Conference on Software Engineering (ICSE)*. Monterey, CA, USA, 1987, págs. 328-338.
- [4] Ken Schwaber y Jeff Sutherland. *The Scrum Guide: The Definitive Guide to Scrum: The Rules of the Game*. Disponible en: <https://scrumguides.org/>. 2020.
- [5] David J. Anderson. *Kanban: Successful Evolutionary Change for Your Technology Business*. Blue Hole Press, 2010.
- [6] Kent Beck y Cynthia Andres. *Extreme Programming Explained: Embrace Change*. 2nd. Addison-Wesley, 2004.
- [7] Len Bass, Paul Clements y Rick Kazman. *Software Architecture in Practice*. 4.^a ed. Addison-Wesley Professional, 2021. ISBN: 978-0136886099.
- [8] Phill Powell y Ian Smalley. *Monolithic architecture vs. microservices: Which works best for you?* Accedido: 2026-05-12. IBM Think. URL: <https://www.ibm.com/think/topics/monolithic-vs-microservices>.
- [9] *What's the Difference Between Monolithic and Microservices Architecture?* Accedido: 2026-05-12. AWS. URL: <https://aws.amazon.com/compare/the-difference-between-monolithic-and-microservices-architecture/>.
- [10] Stephanie Susnjara y Ian Smalley. *Microservices advantages and disadvantages*. Accedido: 2026-05-12. IBM Think, 2025. URL: <https://www.ibm.com/think/insights/microservices-advantages-disadvantages>.
- [11] Manish Hatwalne. *You Want Microservices, But Do You Really Need Them?* Publicado: 28 November 2025; Accedido: 2026-05-12. Docker, 2025. URL: <https://www.docker.com/blog/do-you-really-need-microservices/>.

- [12] Steve Smith. *Architect Modern Web Applications with ASP.NET Core and Azure*. 8th. Microsoft Developer Division, 2023. URL: <https://learn.microsoft.com/en-us/dotnet/architecture/modern-web-apps-azure/>.
- [13] Alistair Cockburn. «Hexagonal Architecture (Ports and Adapters)». En: *HaT Technical Report 2005.02* (2005). URL: <https://alistair.cockburn.us/hexagonal-architecture/>.
- [14] Amazon Web Services. *Hexagonal Architecture Pattern*. 2024. URL: <https://docs.aws.amazon.com/prescriptive-guidance/latest/cloud-design-patterns/hexagonal-architecture.html>.
- [15] Robert C. Martin. *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall, 2018. ISBN: 978-0134494166.
- [16] Eric Evans. *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley Professional, 2003. ISBN: 978-0321125217.
- [17] Martin Fowler. *Patterns of Enterprise Application Architecture*. Addison-Wesley Professional, 2002. ISBN: 978-0321127426.
- [18] Martin Fowler. *Inversion of Control Containers and the Dependency Injection Pattern*. 2004. URL: <https://martinfowler.com/articles/injection.html>.
- [19] Andrew Lock. *Exploring the ASP.NET Core Identity PasswordHasher*. 2017. URL: <https://andrewlock.net/exploring-the-asp-net-core-identity-passwordhasher/> (visitado 21-05-2026).
- [20] World Health Organization - Uppsala Monitoring Centre. *Causality Assessment: WHO-UMC Causality Categories for ADR Reporting*. Accedido: 2026-05-18. URL: <https://nstep.in/index.php/node/1155/>.
- [21] Edgar F. Codd. «A Relational Model of Data for Large Shared Data Banks». En: *Communications of the ACM* 13.6 (1970), págs. 377-387.
- [22] Edgar F. Codd. «Further Normalization of the Data Base Relational Model». En: *Data Base Systems*. Prentice-Hall, 1971, págs. 33-64.
- [23] Christopher J. Date. *An Introduction to Database Systems*. 8th. Pearson, 2012.
- [24] Abraham Silberschatz, Henry F. Korth y S. Sudarshan. *Database System Concepts*. 7th. McGraw-Hill, 2019.
- [25] IBM Corporation. *Administration Guide: Normal Forms*. Accedido: 2026-04-20. 1999. URL: <https://public.dhe.ibm.com/ps/products/db2/info/vr6/htm/db2d0/db2d019.htm>.
- [26] IBM Corporation. *Third Normal Form*. Accedido: 2026-04-20. 2005. URL: <https://publib.boulder.ibm.com/epubs/html/25122710/ddi55.htm>.

- [27] Microsoft. *What's new in .NET 8*. Documentación oficial de Microsoft sobre las novedades y características de .NET 8. 2024. URL: <https://learn.microsoft.com/en-us/dotnet/core/whats-new/dotnet-8> (visitado 31-05-2026) (vid. pág. 1).
- [28] Microsoft. *Entity Framework Core Overview*. Documentación oficial de Entity Framework Core. Describe el mapeo objeto-relacional, las migraciones y la API fluida. 2024. URL: <https://learn.microsoft.com/en-us/ef/core/> (visitado 28-05-2026) (vid. págs. 2, 16).
- [29] Oracle Corporation. *MySQL 8.0 Reference Manual*. Documentación oficial de MySQL. Describe la arquitectura del motor de almacenamiento y las características de integridad referencial. 2024. URL: <https://dev.mysql.com/doc/refman/8.0/en/> (visitado 31-05-2026) (vid. pág. 2).
- [30] Docker Inc. *Docker overview*. 2024. URL: <https://docs.docker.com/get-started/overview/> (visitado 24-05-2026) (vid. pág. 2).
- [31] Railway Corp. *Railway Docs*. 2024. URL: <https://docs.railway.com/> (visitado 24-05-2026) (vid. pág. 2).
- [32] Railway Corp. *Deployments*. 2024. URL: <https://docs.railway.com/deployments/> (visitado 24-05-2026) (vid. pág. 2).
- [33] Vercel Inc. *Vercel Documentation*. 2024. URL: <https://vercel.com/docs> (visitado 24-05-2026) (vid. pág. 3).
- [34] Microsoft. *Using .NET in Visual Studio Code*. 2024. URL: <https://code.visualstudio.com/docs/languages/dotnet> (visitado 24-05-2026) (vid. pág. 3).
- [35] Microsoft. *C# for Visual Studio Code*. 2024. URL: <https://marketplace.visualstudio.com/items?itemName=ms-dotnettools.csharp> (visitado 24-05-2026) (vid. pág. 3).
- [36] GitHub, Inc. *About commits*. 2024. URL: <https://docs.github.com/en/pull-requests/committing-changes-to-your-project/creating-and-editing-commits/about-commits> (visitado 24-05-2026) (vid. pág. 3).
- [37] Scott Chacon y Ben Straub. *Pro Git*. Apress, 2014. Cap. Getting Started - About Version Control. URL: <https://git-scm.com/book/en/v2/Getting-Started-About-Version-Control> (visitado 24-05-2026) (vid. pág. 3).
- [38] SmartBear Software. *Swagger UI*. 2024. URL: <https://swagger.io/tools/swagger-ui/> (visitado 24-05-2026) (vid. pág. 3).
- [39] Microsoft. *Guid.NewGuid Method*. Documentación oficial de .NET. Describe la generación de GUIDs Versión 4 con 122 bits de entropía fuerte. 2026. URL: <https://learn.microsoft.com/en-us/dotnet/api/system.guid.newguid> (vid. pág. 5).

- [40] Microsoft. *IdentityUser<TKey>Class*. Documentación oficial de .NET. Representa un usuario en el sistema Identity con propiedades como Email, PasswordHash, SecurityStamp y LockoutEnd. 2024. URL: <https://learn.microsoft.com/en-us/dotnet/api/microsoft.aspnetcore.identity.identityuser-1> (visitado 25-05-2026) (vid. pág. 6).
- [41] B. Kaliski. *PKCS #5: Password-Based Cryptography Specification Version 2.0*. Inf. téc. 2898. Estándar que define PBKDF2. Especifica la derivación de claves a partir de contraseñas mediante la aplicación iterativa de una función pseudoaleatoria (como HMAC-SHA256) combinada con un salt. IETF, 2000. URL: <https://datatracker.ietf.org/doc/html/rfc2898> (visitado 25-05-2026) (vid. pág. 6).
- [42] H. Krawczyk, M. Bellare y R. Canetti. *HMAC: Keyed-Hashing for Message Authentication*. Inf. téc. 2104. Especificación oficial del mecanismo HMAC. Define el algoritmo para autenticación de mensajes mediante funciones hash criptográficas como SHA-256. IETF, 1997. URL: <https://datatracker.ietf.org/doc/html/rfc2104> (visitado 25-05-2026) (vid. pág. 7).
- [43] Microsoft. *PasswordHasher<TUser>Class*. Documentación oficial de ASP.NET Core Identity. Implementa PBKDF2 con HMAC-SHA256, salt de 128 bits y 100,000 iteraciones por defecto para el hashing de contraseñas. 2024. URL: <https://learn.microsoft.com/en-us/dotnet/api/microsoft.aspnetcore.identity.passwordhasher-1> (visitado 25-05-2026) (vid. pág. 7).
- [44] Microsoft. *ASP.NET Core Identity PasswordHasher*. Repositorio oficial en GitHub. Contiene la implementación del formato de hash: Base64(salt).Base64(subkey). 2024. URL: <https://github.com/aspnet/AspNetIdentity/blob/main/src/Microsoft.AspNetCore.Identity.Core/PasswordHasher.cs> (visitado 25-05-2026) (vid. pág. 7).
- [45] OWASP Foundation. *Password Storage Cheat Sheet*. Guía oficial de OWASP para almacenamiento seguro de contraseñas. Recomienda PBKDF2-SHA256 con 600,000 iteraciones para aplicaciones en producción. 2024. URL: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html (visitado 25-05-2026) (vid. págs. 7, 8).
- [46] Microsoft. *SecurityStamp Validation in ASP.NET Core Identity*. Documentación oficial. Explica que el SecurityStamp se regenera al cambiar credenciales e invalida sesiones y tokens anteriores. 2024. URL: <https://learn.microsoft.com/en-us/aspnet/core/security/authentication/identity-configuration> (visitado 25-05-2026) (vid. pág. 7).

- [47] National Institute of Standards and Technology. *Advanced Encryption Standard (AES)*. Inf. téc. FIPS PUB 197. Estándar oficial del NIST que define el algoritmo de cifrado simétrico AES con claves de 128, 192 y 256 bits y el modo de operación CBC. NIST, 2001. URL: <https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.197.pdf> (visitado 25-05-2026) (vid. pág. 8).
- [48] Microsoft. *Token providers in ASP.NET Core Identity*. Documentación oficial. Describe los proveedores de tokens para confirmación de correo, restablecimiento de contraseña y autenticación de dos factores. 2024. URL: <https://learn.microsoft.com/en-us/aspnet/core/security/authentication/identity-configuration> (visitado 25-05-2026) (vid. pág. 8).
- [49] Microsoft. *ASP.NET Core Data Protection Overview*. Documentación oficial. Describe la API de protección de datos usada para cifrar y firmar tokens, con gestión automática de claves y rotación. 2024. URL: <https://learn.microsoft.com/en-us/aspnet/core/security/data-protection/introduction> (visitado 25-05-2026) (vid. pág. 8).
- [50] Microsoft. *Configure ASP.NET Core Identity*. Documentación oficial. Cubre la configuración de IdentityOptions: bloqueo, contraseñas, tokens, cookies y validación de SecurityStamp. 2024. URL: <https://learn.microsoft.com/en-us/aspnet/core/security/authentication/identity-configuration> (visitado 25-05-2026) (vid. pág. 8).
- [51] Kirk Larkin, Steve Smith y Brandon Dahler. *Dependency injection in ASP.NET Core*. 2026. URL: <https://learn.microsoft.com/en-us/aspnet/core/fundamentals/dependency-injection> (vid. pág. 10).
- [52] Bertrand Meyer. *Object-Oriented Software Construction*. Prentice Hall, 1988 (vid. pág. 11).
- [53] Martin Fowler. *Data Mapper*. Descripción del patrón Data Mapper, que transfiere datos entre objetos y la base de datos manteniéndolos independientes. 2003. URL: <https://martinfowler.com/eaCatalog/dataMapper.html> (visitado 30-05-2026) (vid. pág. 13).
- [54] Pomelo Foundation. *Pomelo.EntityFrameworkCore.MySql*. Proveedor de Entity Framework Core para MySQL. Documentación oficial del paquete Pomelo.EntityFrameworkCore.MySql. 2024. URL: <https://github.com/PomeloFoundation/Pomelo.EntityFrameworkCore.MySql> (visitado 28-05-2026) (vid. pág. 16).
- [55] Martin Fowler. *Unit of Work*. Descripción del patrón Unit of Work: mantiene el registro de cambios durante una transacción de negocio y los persiste de forma atómica. 2005. URL: <https://martinfowler.com/eaCatalog/unitOfWork.html> (visitado 28-05-2026) (vid. pág. 18).

- [56] QuestPDF. *QuestPDF Documentation*. Documentación oficial de QuestPDF. Librería para generar documentos PDF mediante API fluida en C#. 2024. URL: <https://www.questpdf.com/> (visitado 28-05-2026) (vid. pág. 20).
- [57] MassTransit. *MassTransit Documentation*. Documentación oficial de MassTransit. Framework para mensajería asíncrona con soporte para RabbitMQ. 2024. URL: <https://masstransit.io/documentation> (visitado 28-05-2026) (vid. pág. 20).
- [58] Microsoft. *Background tasks with hosted services in ASP.NET Core*. Documentación oficial sobre servicios en segundo plano en ASP.NET Core mediante BackgroundService. 2026. URL: <https://learn.microsoft.com/en-us/aspnet/core/fundamentals/host/hosted-services> (visitado 28-05-2026) (vid. pág. 21).
- [59] Roy Thomas Fielding. «Architectural Styles and the Design of Network-based Software Architectures». Tesis doctoral que define el estilo arquitectónico REST. Establece los principios de comunicación sin estado, identificación de recursos mediante URLs y uso de verbos HTTP estándar. Tesis doct. University of California, Irvine, 2000. URL: https://www.ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm (visitado 28-05-2026) (vid. pág. 22).
- [60] Microsoft. *Handle errors in ASP.NET Core*. Documentación oficial sobre manejo de errores en ASP.NET Core, incluyendo middleware de excepciones y páginas de error. 2024. URL: <https://learn.microsoft.com/en-us/aspnet/core/fundamentals/error-handling> (visitado 28-05-2026) (vid. pág. 24).
- [61] Microsoft. *Model validation in ASP.NET Core MVC*. Documentación oficial sobre validación de modelos y configuración de ApiBehaviorOptions para personalizar respuestas de validación. 2024. URL: <https://learn.microsoft.com/en-us/aspnet/core/mvc/models/validation> (visitado 28-05-2026) (vid. pág. 25).
- [62] M. Jones, J. Bradley y N. Sakimura. *JSON Web Token (JWT)*. Inf. téc. 7519. Estándar que define la estructura, claims y firma de los JSON Web Tokens. IETF, 2015. URL: <https://datatracker.ietf.org/doc/html/rfc7519> (visitado 25-05-2026) (vid. pág. 26).
- [63] Microsoft. *RandomNumberGenerator Class*. Documentación oficial de .NET sobre la generación de números aleatorios criptográficamente seguros. 2024. URL: <https://learn.microsoft.com/en-us/dotnet/api/system.security.cryptography.randomnumbergenerator> (visitado 25-05-2026) (vid. pág. 28).
- [64] OWASP Foundation. *HttpOnly*. Documentación de OWASP sobre el atributo HttpOnly. Explica cómo impide que JavaScript acceda a las cookies, mitigando ataques XSS. 2024. URL: <https://owasp.org/www-community/HttpOnly> (visitado 26-05-2026) (vid. pág. 28).

- [65] OWASP Foundation. *Secure Cookie Attribute*. Guía de OWASP sobre atributos de seguridad en cookies. Explica HttpOnly como defensa contra XSS, Secure para transmisión exclusiva sobre HTTPS, y SameSite para controlar el envío en solicitudes cross-site. 2024. URL: <https://owasp.org/www-community/controls/SecureCookieAttribute> (visitado 26-05-2026) (vid. pág. 28).
- [66] M. Jones y D. Hardt. *The OAuth 2.0 Authorization Framework: Bearer Token Usage*. Inf. téc. 6750. Estándar que define el uso del encabezado Authorization: Bearer para enviar tokens de acceso. IETF, 2012. URL: <https://datatracker.ietf.org/doc/html/rfc6750> (visitado 25-05-2026) (vid. pág. 29).
- [67] Friendly Captcha GmbH. *Friendly CAPTCHA Documentation*. Documentación oficial de Friendly CAPTCHA. Describe el funcionamiento del proof-of-work y la API de verificación de tokens. 2024. URL: <https://docs.friendlycaptcha.com/> (visitado 31-05-2026) (vid. pág. 31).
- [68] Microsoft. *Rate limiting middleware in ASP.NET Core*. Documentación oficial sobre la configuración del middleware de rate limiting en ASP.NET Core. Describe las políticas de ventana fija, ventana deslizante y limitación por IP. 2024. URL: <https://learn.microsoft.com/en-us/aspnet/core/performance/rate-limit> (visitado 27-05-2026) (vid. págs. 32, 33).
- [69] Centro para el Control Estatal de Medicamentos, Equipos y Dispositivos Médicos. *Informe anual de farmacovigilancia. Cuba, 2020*. Accedido el 3 de mayo de 2026. 2020. URL: https://www.cecmed.cu/sites/default/files/adjuntos/vigilancia/farmacov/FV_para_informe_anual_2020.pdf (vid. págs. 39, 40).
- [70] Organización Panamericana de la Salud. *Manual de vigilancia de eventos supuestamente atribuibles a la vacunación o inmunización en la Región de las Américas*. Versión oficial adaptada en español de la obra original en inglés: Global manual on surveillance of adverse events following immunization (WHO, 2016). Organización Panamericana de la Salud. Washington, D.C., 2021. ISBN: 978-92-75-32386-1. DOI: 10.37774/9789275323861 (vid. pág. 41).
- [71] Celeste Sánchez G. et al. *Vigilancia de medicamentos en Cuba. Desarrollo actual y nuevos retos*. Accedido el 3 de mayo de 2026. 2010. URL: <https://www.paho.org/sites/default/files/CUB-VIGILANCIA-DE-MEDICAMENTOS-EN-CUBA.pdf> (vid. págs. 41, 42).
- [72] Giset Jiménez López e Ismary Alfonso Orta. *Normas y procedimientos de trabajo del sistema cubano de farmacovigilancia*. Accedido el 3 de mayo de 2026. 2012. URL: <https://files.sld.cu/cdfc/files/2012/10/normas-y-procedimientos-2012.pdf> (vid. págs. 42, 44, 45, 47, 48, 51).

- [73] Centro para el Control Estatal de Medicamentos, Equipos y Dispositivos Médicos (CECMED). «Disposición Reguladora No. 00-515». En: *Boletín del CECMED* XXVI.00-515 (oct. de 2025). Edición Ordinaria. Consultada la página 22 donde se encuentra el modelo 84-30-2. ISSN: 1684-1832. URL: <https://www.cecmed.cu/sites/default/files/adjuntos/ambitor/AR%20No.%2000-515.pdf> (vid. pág. 50).
- [74] Andrew Hunt y David Thomas. *The Pragmatic Programmer: From Journeyman to Master*. Addison-Wesley, 1999. ISBN: 978-0135957059 (vid. pág. 61).
- [75] Robert C. Martin. «Design Principles and Design Patterns». En: (). URL: https://staff.cs.utu.fi/~jounsmmed/doos_06/material/DesignPrinciplesAndPatterns.pdf (vid. pág. 62).
- [76] Michael Feathers. *The SOLID Principles*. Correspondencia con Robert C. Martin donde se acuña el acrónimo SOLID. 2004 (vid. pág. 62).